TruthTable++: Verifiable Query Engine with complete TPC-H support

Alireza Shirzad
Upenn

Bharath Namboothiry
Upenn

Ryan Marcus
Upenn

Pratyush Mishra
Upenn

June 29, 2026

Abstract

In the paper we present something cool

Contents

1 Introduction

[Ali: no other prior works support string operations] [Ali: some statistics: X percent of the queries out there have string predicates, Y percent of the TPCH queries have string predicates. A verifiable database without string support loses all these applciations, something like that]

2 Preliminaries

We introduce notation and cryptographic background required to understand TruthTable++'s protocols.

Boolean hypercube. The μ-dimensional Boolean hypercube $\mathscr{H}_\mu$ is the set $\{0,1\}^\mu$. It has size $n = 2^\mu$. We will commonly interpret elements of $\mathscr{H}_\mu$ as integers in the range $[0, n-1]$.

Polynomials. We write $p \in \mathbb{F}^{\leq d}[X_1, \ldots, X_\mu]$ (or, concisely, $p \in \mathbb{F}^{\leq d}[\mathbf{X}]$) to denote a polynomial in μ variables over the field $\mathbb{F}$, where each variable has degree at most d.

Multilinear extensions. Let $v \in \mathbb{F}^N$ be a vector of field elements with $N = 2^\mu$ for some $\mu \in \mathbb{N}$. The *multilinear extension* $\tilde{v}$ of v is the unique multilinear polynomial whose evaluations at the i-th point in the μ-dimensional boolean hypercube equal the i-th element of v. We now recall some useful multivariate polynomials.

- The *index polynomial* $\mathcal{I}_\mu(X) := \sum_{i=1}^{\mu} 2^{i-1} X_i$ maps the i-th point in $\mathscr{H}_\mu$ to the value $i-1$.
- The *equality* polynomial $\mathsf{Eq}(X,Y) := \prod_{i=1}^{\mu}((1-X_i)(1-Y_i) + X_iY_i)$ satisfies the property that, for $x, y \in \mathscr{H}_\mu$, $\mathsf{Eq}(x,y)$ equals 1 if $x = y$, and equals 0 otherwise.
- The *s-contiguous* polynomial $\mathcal{C}_{\mu,s}(X) := \sum_{i=1}^{\mu} s_i(1-X_i)\prod_{j=1}^{i-1}(s_jX_j + (1-s_j)(1-X_j))$ is 1 for the first s points in $\mathscr{H}_\mu$ and 0 elsewhere. Here, s_i is the i-th most significant bit of s. We provide a derivation of this polynomial in Section A.1.

Polynomial oracles. Algorithms with *oracle* access to a polynomial p (denoted $[\![p]\!]$) can query p at any point $x \in \mathbb{F}^\mu$ to obtain the evaluation $p(x)$.

Polynomial commitments. A polynomial commitment scheme is a cryptographic primitive that allows one to generate a succinct commitment cm_p to a polynomial p, and later reveal its evaluation at specific points, along with a proof that the evaluation is correct [KZG10].

SNARKs via polynomials. TruthTable++ follows the now-standard methodology for constructing SNARKs by combining *Polynomial Interactive Oracle Proofs* (PIOPs) with *Polynomial Commitment* (PC) schemes [BFS20; CHMMVW20]. A PIOP is an idealized interactive proof between a prover $\mathbf{P}$ and a verifier $\mathbf{V}$, in which the prover sends polynomial oracles and the verifier queries these oracles before deciding acceptance.

All protocol design in TruthTable++ occurs at the PIOP layer. We build PIOPs that prove correct execution of relational operators over database tables, and build these protocols compositionally using polynomial interactive oracle *reductions* [BCGGRS19; BMNW25; BMMS25], which interactively reduce complex claims to simpler algebraic subclaims. As shown in Section 4, each operator-level protocol ultimately reduces to a small collection of low-level algebraic claims, which are proven using standard PIOPs from the literature. For completeness, we briefly recall the three protocols frequently used in TruthTable++, deferring formal definitions to Section A.2.

- **Sumcheck** proves claims of the form $\sum_{x \in \mathscr{H}_\mu} p(x) = s$ for a polynomial p and field element s.
- **Zerocheck** proves that a polynomial p vanishes on the Boolean hypercube, i.e., $p(x) = 0$ for all $x \in \mathscr{H}_\mu$.
- **Rational Sumcheck** proves claims of the form $\sum_{x \in \mathscr{H}_\mu} \frac{p(x)}{q(x)} = s$ for polynomials p, q, under the promise that $q(x) \neq 0$ for all $x \in \mathscr{H}_\mu$.

To obtain a SNARK, we compile these PIOPs using a polynomial commitment scheme [CHMMVW20; BFS20]. Intuitively, this compilation emulates the polynomial oracles in the PIOP model: each polynomial oracle sent by the prover is replaced by a *polynomial commitment*, and each oracle query is replaced by a proof that the committed polynomial evaluates to the claimed value at the verifier's chosen point [CHMMVW20; GWC19]. Since this compilation applies generically, we treat it as a black box and focus exclusively on

designing efficient PIOPs for relational operators; the resulting SNARK inherits its efficiency and soundness directly from the underlying PIOPs and commitment scheme.

3 Arithmetization

Let c be a string column in the database. We encode c in as a tuple of 8 columns, $(\mathsf{h}, \mathsf{l}, \mathsf{a_{hb}}, \mathsf{c}, \mathsf{src}, \mathsf{o}, \mathsf{a_c}, \mathsf{s_b})$, with the following semantics:

- Hash column h is of the same length as c containing the hash of each string in c.
- Length column l is of the same length as c, with $\mathsf{l}[i]$ being the length $|\mathsf{c}[i]|$ of the i-th string in c.
- Data column c is of the size $\sum_{s \in \mathsf{c}} |s|$, containing the concatenation of all strings in c.
- Source column src is of the same length as c, with $\mathsf{src}[j]$ being the index i of the string in c that the j-th character of c belongs to.
- Within-word index column o is of the same length as c, with $\mathsf{o}[j]$ being the position of the j-th character of c inside its own string, counting from 0. Equivalently, o resets to 0 at every string boundary and increments by one across the characters of each string, so it reads $0, 1, 2,\ 0, 1, 2, 3, 4,\ 0, 1, 2, 3, 4, 5,\ 0, 1, \ldots$ over consecutive strings.
- The data activator column $\mathsf{a_c}$ is of the same length as c, with $\mathsf{a_c}[i]$ being 1 if the i-th character in c is an active chatacter of the column c.
- The hash/boundry activator column $\mathsf{a_{hb}}$ is of the same length as c, with $\mathsf{a_{hb}}[i]$ being 1 if the i-th string in c is active.
- The boundary selector column $\mathsf{s_b}$ is of the same length as c, with $\mathsf{s_b}[j]$ being 1 if the j-th character of c is the first character of a string (a string boundary), and 0 otherwise.

4 PIOP Toolbox

TruthTable++ builds on a small toolbox of low-level PIOPs and relations. Most of these are standard or lightly adapted from prior work, so we defer their relations and constructions to Section B and recall them inline as needed. Here we present only the Bitwise AND check, which illustrates the truth-table lookup technique that underlies several of our string protocols.

4.1 Bitwise AND

The *Bitwise AND check* takes three columns $\mathsf{c}_1, \mathsf{c}_2, \mathsf{c}_3$ containing n-bit values and proves that every entry of c_3 is the bitwise AND of the corresponding entries in c_1 and c_2. We package this as the relation

$$\mathscr{R}_{\mathsf{BitwiseAND}} = \{\, \mathbb{x} = n,\ \mathbb{y} = (\llbracket \tilde{\mathsf{c}_1} \rrbracket, \llbracket \tilde{\mathsf{c}_2} \rrbracket, \llbracket \tilde{\mathsf{c}_3} \rrbracket),\ \mathbb{w} = (\mathsf{c}_1, \mathsf{c}_2, \mathsf{c}_3) \mid \forall i,\ \mathsf{c}_3[i] = \mathsf{c}_1[i] \wedge \mathsf{c}_2[i] \,\}\,.$$

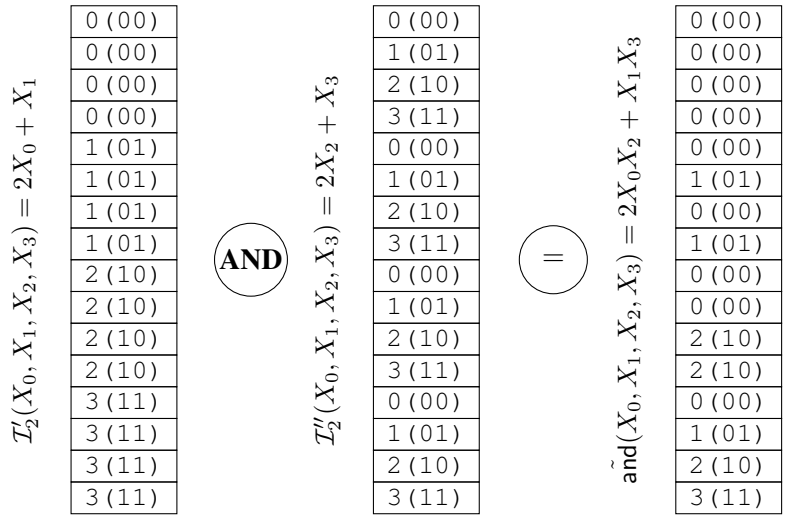

Figure 1: The arithmetized truth-table for the bitwise AND function on n=2 bit inputs.

To convince the verifier that each row of c_3 is the bitwise AND of the corresponding rows of c_1 and c_2, the prover will convince the verifier that the concatenated rows of $\mathsf{c}_1, \mathsf{c}_2, \mathsf{c}_3$ can be looked up in a truth table encoding the bitwise AND function. More concretely,

PIOP 1: BitwiseAND

1. $\mathbf{P}$ and $\mathbf{V}$ invoke Range Check on each of the three columns to check that they are all n-bit.
2. $\mathbf{V}$ samples three random field elements r_1, r_2, r_3 and sends them to $\mathbf{P}$.
3. $\mathbf{P}$ and $\mathbf{V}$ invoke Lookup Check to check that the concatenated column $\mathsf{c}' = r_1\mathsf{c}_1 + r_2\mathsf{c}_2 + r_3\mathsf{c}_3$ can be looked up in the column $\mathsf{c}'' = r_1\mathcal{I}'_n + r_2\mathcal{I}''_n + r_3\tilde{o}$ where

$$\mathcal{I}'_n(X_0, \ldots, X_{n-1}) = \sum_{i=0}^{n-1} 2^{n-1-i} X_i,$$

$$\mathcal{I}''_n(X_0, \ldots, X_{n-1}) = \sum_{i=0}^{n-1} 2^{n-1-i} X_{n+i},$$

$$\tilde{\mathsf{and}}(X_0, \ldots, X_{2n-1}) = \sum_{i=0}^{n-1} 2^{n-1-i} X_i X_{n+i}.$$

4.2 Activator Consistency Check

As described in Section 3, a string column carries two activators: $\mathsf{a_h}$, marking the active strings on the hash polynomial, and $\mathsf{a_c}$, marking the active characters on the data polynomial. The protocols that follow repeatedly have the prover send a fresh pair of such activators, so the verifier's first task is to confirm that the pair is mutually consistent: every character of an active string must be active, and every character of an inactive string inactive. We package this as the relation

$$\mathscr{R}_{\mathsf{ActCons}} = \{\, \mathbb{x} = \bot,\ \mathbb{y} = ([\![\tilde{\mathsf{a_c}}]\!], [\![\tilde{\mathsf{a_h}}]\!]),\ \mathbb{w} = (\mathsf{a_c}, \mathsf{a_h}) \mid \mathsf{a_c}[j] = \mathsf{a_h}[i] \text{ for every character } j \text{ of string } i \,\}.$$

We design a PIOP for this relation that relies on the following observations: grouping the rows $(\mathsf{src}, \mathsf{a_c})$ by src and summing $\mathsf{a_c}$ within each group must reproduce l on the active strings (those marked by $\mathsf{a_h}$). Hence, treating $(\mathsf{src}, \mathsf{a_c})$ as an input table grouped by its key and $(\mathsf{ind}, \mathsf{l})$ with activator $\mathsf{a_h}$ as the grouped output, We can invoke the `Group By Sum` protocol from [NSSMM26] which is a single KeyedSumcheck invocation.

PIOP 2: Activator Consistency Check

Inputs: $[\![\tilde{\mathsf{src}}]\!], [\![\tilde{\mathsf{a_c}}]\!], [\![\tilde{\mathsf{l}}]\!], [\![\tilde{\mathsf{a_h}}]\!]$.

1. $\mathbf{P}$ and $\mathbf{V}$ define the table L with key column $\mathsf{L.key} = \mathsf{src}$, value column $\mathsf{L.val} = \mathbf{1}$, and activator $\mathsf{L.a} = \mathsf{a_c}$.
2. $\mathbf{P}$ and $\mathbf{V}$ define the table R with key column $\mathsf{R.key} = \mathsf{ind}$, value column $\mathsf{R.val} = \mathsf{l}$, and activator $\mathsf{R.a} = \mathsf{a_h}$.
3. $\mathbf{P}$ and $\mathbf{V}$ invoke KeyedSumcheck on L and R, which forces $\sum_{j:\,\mathsf{src}[j]=i} \mathsf{a_c}[j] = \mathsf{a_h}[i] \cdot \mathsf{l}[i]$ for every key i.

4.3 Broadcast Check

The *Broadcast Check* takes a string-level column x (one entry per string, over the hash hypercube) and a character-level column $\mathsf{x'}$, and proves that $\mathsf{x'}$ copies each string's value to all of its characters, i.e. every character carries the value of its own string. We package this as the relation

$$\mathscr{R}_{\mathsf{Bcast}} = \left\{ \mathbb{x} = \bot,\ \mathbb{y} = ([\![\tilde{\mathsf{src}}]\!], [\![\tilde{\mathsf{x}}]\!], [\![\tilde{\mathsf{x'}}]\!]),\ \mathbb{w} = (\mathsf{src}, \mathsf{x}, \mathsf{x'}) \;\middle|\; \mathsf{x'}[c] = \mathsf{x}[\mathsf{src}[c]] \text{ for every character } c \right\}.$$

Since the string indices ind are distinct, the pair $(\mathsf{src}[c], \mathsf{x'}[c])$ equals the string row $(\mathsf{ind}[i], \mathsf{x}[i])$ exactly for $i = \mathsf{src}[c]$. Encoding each pair as a single field element $\mathsf{key} + r \cdot \mathsf{val}$ with a random challenge r collapses the check to one lookup of the character pairs into the string pairs: by Schwartz–Zippel, $\mathsf{src}[c] + r\,\mathsf{x'}[c] \in \{\mathsf{ind}[i] + r\,\mathsf{x}[i]\}_i$ at a random r forces $(\mathsf{src}[c], \mathsf{x'}[c]) = (\mathsf{src}[c], \mathsf{x}[\mathsf{src}[c]])$, hence $\mathsf{x'}[c] = \mathsf{x}[\mathsf{src}[c]]$.

PIOP 3: Broadcast Check

Inputs: $[\![\tilde{\mathsf{src}}]\!], [\![\tilde{\mathsf{ind}}]\!], [\![\tilde{\mathsf{x}}]\!], [\![\tilde{\mathsf{x'}}]\!]$.

1. $\mathbf{V}$ samples a random $r \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathbf{P}$.
2. $\mathbf{P}$ and $\mathbf{V}$ invoke Lookup Check on $(\mathbf{1},\ \mathsf{src} + r\,\mathsf{x'})$ and $(\mathbf{1},\ \mathsf{ind} + r\,\mathsf{x})$, proving $(\mathbf{1},\ \mathsf{src} + r\,\mathsf{x'}) \sqsubseteq (\mathbf{1},\ \mathsf{ind} + r\,\mathsf{x})$.

4.4 Length Filtering

In every pattern-matching filter — prefix, suffix, and substring — a string can match only if it is at least as long as the pattern. Any shorter string is therefore a guaranteed non-match and can be discarded before the filter runs. As we will see in Sections 5.1.1 and 7.1, this is not merely an optimization: length filtering is necessary for the pattern-matching protocols to remain complete and sound. We define the length filtering relation as follows

$$\mathscr{R}_{\mathsf{LenFilter}} = \Big\{ \mathbb{x} = k,\ \mathbb{y} = (\tilde{[\![\mathsf{l}]\!]}, \tilde{[\![\mathsf{a}_\mathsf{h}]\!]}, \tilde{[\![\mathsf{a}_\mathsf{c}]\!]}, \tilde{[\![\mathsf{a}'_\mathsf{h}]\!]}, \tilde{[\![\mathsf{a}'_\mathsf{c}]\!]}),\ \mathbb{w} = (\mathsf{l}, \mathsf{a}_\mathsf{h}, \mathsf{a}_\mathsf{c}, \mathsf{a}'_\mathsf{h}, \mathsf{a}'_\mathsf{c}) \Big|\ \mathsf{a}'_\mathsf{h}[i] = 1 \iff \big(\mathsf{a}_\mathsf{h}[i] = 1 \text{ and } \mathsf{l}[i] \geq k\big) \text{ and } \mathsf{a}'_\mathsf{c}[j] = \mathsf{a}'_\mathsf{h}[i] \text{ for every character } j \text{ of string } i \Big\}.$$

PIOP 4: Length Filtering Check

Inputs: $\tilde{[\![\mathsf{src}]\!]}, \tilde{[\![\mathsf{l}]\!]}, \tilde{[\![\mathsf{a}_\mathsf{h}]\!]}, \tilde{[\![\mathsf{a}_\mathsf{c}]\!]}$, threshold k.

1. (*Activator validity check*)
   (a) $\mathbf{P}$ and $\mathbf{V}$ invoke Booleanity Check on a'_h and on a'_c.
   (b) $\mathbf{P}$ and $\mathbf{V}$ invoke Activator Consistency Check on $(\mathsf{a}'_\mathsf{c}, \mathsf{a}'_\mathsf{h})$.
2. (*Containment*) $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $\mathsf{a}'_\mathsf{h} \cdot (1 - \mathsf{a}_\mathsf{h})$.
3. (*False negative check*) $\mathbf{P}$ and $\mathbf{V}$ invoke the Negative Sign Check on the column $(\mathsf{a}_\mathsf{h} \cdot (1 - \mathsf{a}'_\mathsf{h}), \mathsf{l} - k)$.
4. (*False Positive check*) $\mathbf{P}$ and $\mathbf{V}$ invoke the Non-Negative Sign Check on the column $(\mathsf{a}'_\mathsf{h}, \mathsf{l} - k)$.

Proof. Every kept string is active (containment) and at least k long (no false positives), and every active string of length at least k is kept (no false negatives), so the kept strings are exactly the active strings of length $\geq k$; by validity, a'_c is the matching character activator. This is precisely $\mathscr{R}_{\mathsf{LenFilter}}$. Completeness is immediate: the honest output — the input activators with the short strings switched off — is boolean, consistent, contained in a_h, and passes both length checks. □

5 PIOPs for Wild-Card Pattern Matching

In this section we develop PIOPs for SQL *wild-card* pattern matching — the predicates written with the `LIKE` operator. A `LIKE` pattern may use two kinds of wild-card: the multi-character `%`, which matches any (possibly empty) run of characters, and the single-character `_`, which matches exactly one character. Figure 2 shows the taxonomy of patterns we support: three kinds — single-character, multi-character, and mixed — where the single- and multi-character kinds each come in a single-factor and a multi-factor mode.

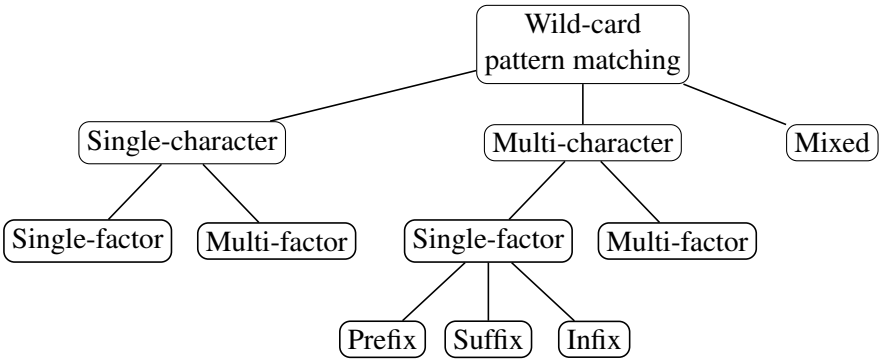

Figure 2: Taxonomy of wild-card pattern matching

5.1 Multi-Character

The most common kind of string pattern matching in natural SQL workloads is a multi-character pattern matching which is of the form $s_1\%s_2\%\ldots\%s_k$, where each s_i is a literal string (a factor) and % matches an arbitrary-length run of characters — so the predicate holds iff the factors $s_1\%s_2\%\ldots\%s_k$ occur in order within the value, with the leading and trailing % (or their absence) determining whether the match is anchored to the start and/or end of the string.

Depending on the number of factors, we call the expressions as a single-factor or multi-factor pattern. We start by describing protocols for single-factor multi-character pattern matching in Section 5.1.1, then lift the construction to a multi-factor setting in Section 7.1.1.

5.1.1 Single-Factor

A single-factor multi-character pattern matches a single literal factor against the value, and—depending on where the % wild-cards are placed—comes in three forms:

- *prefix* (`'Promo%'`): the factor must appear at the start of the value;
- *suffix* (`'%BRASS'`): the factor must appear at the end of the value;
- *infix* (`'%green%'`): the factor may appear anywhere in the value.

We present a single general protocol (SFMC Pattern Matching) that proves all three of prefix, suffix and infix matching against a length-k pattern str. We first give the intuition for the infix case, which is the hardest, and then explain why prefix and suffix fall out as strictly easier special cases.

Proving infix. The verifier receives a new pair of activators $\mathsf{a}_\mathsf{h}^{\mathsf{new}}, \mathsf{a}_\mathsf{c}^{\mathsf{new}}$ and must check that, relative to the old activators $\mathsf{a}_\mathsf{h}^{\mathsf{old}}, \mathsf{a}_\mathsf{c}^{\mathsf{old}}$, they keep exactly the words containing str and switch off all the rest. The protocol does this in four steps. (i) *Validity.* The verifier first checks that the new activators are well-formed: boolean, mutually consistent (a word is on iff all of its characters are on), and contained in the old activators (no

inactive word is revived). (ii) *Localization.* A match of str can begin at any of the $\mathsf{l} - k + 1$ positions of a word, so naively the verifier would have to inspect a whole window of k characters at every position. To collapse each window to a single slot, the verifier draws random coefficients $r_0, \ldots, r_{k-1}$ and the parties form the *window fingerprint* $\mathsf{wf} := \sum_\delta r_\delta \, \mathsf{c}^{(\delta)}$ (a random linear combination of the rotations of the character column) and the *pattern fingerprint* $\mathsf{pf} := \sum_\delta r_\delta \, \mathsf{str}[\delta]$. Their difference $\mathsf{diff} := \mathsf{wf} - \mathsf{pf}$ then has $\mathsf{diff}[p] = 0$ exactly when the length-k window starting at p equals str (up to a negligible fingerprint collision). The whole question now reduces to where diff vanishes. (iii) *No false negatives.* Every word that is switched off must really fail to contain str. The verifier asks that diff have *no* zero among the candidate slots of those words — a single no-zero check — which certifies that the pattern occurs nowhere inside them. (iv) *No false positives.* Every word that is kept on must really contain str, i.e. diff must have *at least one* zero among its candidate slots. The difficulty is that this zero can sit anywhere in the word, so the prover points it out: it sends a selector that is 1 on one matching slot of each kept word and 0 elsewhere, and the verifier checks that diff indeed vanishes there. A short series of checks (booleanity, no duplicates, and a counting argument) then confirms the selector is honest — it marks one genuine slot per kept word and nothing else.

Prefix and suffix. These are the infix protocol with one simplification: the matching slot is no longer ambiguous. A prefix match can only begin at a word's first character and a suffix match only at its last, so the verifier already knows where diff must vanish. The prover therefore sends no selector and skips all of the checks that would establish its validity.

PIOP 5: Single-Factor Multi-Character (SFMC) Pattern Matching

1. (*Activator validity check*)
   (a) $\mathbf{P}$ and $\mathbf{V}$ invoke Booleanity Check on $\mathsf{a}_\mathsf{c}^{\mathsf{new}}$ and on $\mathsf{a}_\mathsf{h}^{\mathsf{new}}$.
   (b) $\mathbf{P}$ and $\mathbf{V}$ invoke Activator Consistency Check on $(\mathsf{a}_\mathsf{c}^{\mathsf{new}}, \mathsf{a}_\mathsf{h}^{\mathsf{new}})$.
2. (*Filtering by length*) $\mathbf{P}$ computes $\mathsf{a}_\mathsf{h}^{\mathsf{old}'}$ and $\mathsf{a}_\mathsf{c}^{\mathsf{old}'}$ which is the result of filtering the strings with length less than k, then $\mathbf{P}$ and $\mathbf{V}$ invoke Length Filtering Check $(\mathsf{a}_\mathsf{h}^{\mathsf{old}}, \mathsf{a}_\mathsf{c}^{\mathsf{old}}, \mathsf{a}_\mathsf{h}^{\mathsf{old}'}, \mathsf{a}_\mathsf{c}^{\mathsf{old}'})$ with threshold k.
3. (*Containment*) $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on $\mathsf{a}_\mathsf{h}^{\mathsf{new}} \cdot (1 - \mathsf{a}_\mathsf{h}^{\mathsf{old}'})$.
4. (*Rotated character columns*) $\mathbf{P}$ sends $\mathsf{c}^{(1)}, \ldots, \mathsf{c}^{(k-1)}$, each $\mathsf{c}^{(\delta)}$ a rotation of $\mathsf{c}^{(\delta-1)}$ by -1 (so $\mathsf{c}^{(\delta)}[p] = \mathsf{c}[p+\delta]$) and $\mathsf{c}^{(0)} := \mathsf{c}$. Then the parties invoke Rotation Check on each $(\mathsf{c}^{(\delta-1)}, \mathsf{c}^{(\delta)}, -1)$.
5. (*Window fingerprinting*) $\mathbf{V}$ samples $r_0, \ldots, r_{k-1} \xleftarrow{\$} \mathbb{F}$ and sends them to $\mathbf{P}$.
   (a) $\mathbf{P}$ and $\mathbf{V}$ form the windowed fingerprint of the character polynomial as $\mathsf{wf} := \sum_{\delta=0}^{k-1} r_\delta \, \mathsf{c}^{(\delta)}$.
   (b) $\mathbf{P}$ and $\mathbf{V}$ form the fingerprint of the pattern as $\mathsf{pf} := \sum_{\delta=0}^{k-1} r_\delta \, \mathsf{str}[\delta]$
   (c) $\mathbf{P}$ and $\mathbf{V}$ form the difference polynomial as $\mathsf{diff} := \mathsf{wf} - \mathsf{pf}$
6. (*Attention mask*) $\mathbf{P}$ and $\mathbf{V}$ fix the attention mask $\mathsf{att\_mask}$ as follows:
   (a) if the pattern is *prefix*, they set $\mathsf{att\_mask} := \mathsf{a}_\mathsf{c}^{\mathsf{old}'} \cdot \mathsf{s}_\mathsf{b}$
   (b) else if the pattern is *suffix*, $\mathbf{P}$ computes $\mathsf{att\_mask} := \rho_{-k}(\mathsf{a}_\mathsf{c}^{\mathsf{old}'} \cdot \mathsf{s}_\mathsf{b})$ and sends it to the verifier. Then the parties invoke Rotation Check on $(\mathsf{a}_\mathsf{c}^{\mathsf{old}'} \cdot \mathsf{s}_\mathsf{b},\ \mathsf{att\_mask},\ -k)$.
   (c) else (where the pattern is *infix*), $\mathbf{P}$ sends the rotated boundary selectors $\mathsf{s}_\mathsf{b}{}^{(1)}, \ldots, \mathsf{s}_\mathsf{b}{}^{(k-1)}$ ($\mathsf{s}_\mathsf{b}{}^{(0)} := \mathsf{s}_\mathsf{b}$, each a rotation of the previous by -1). Then the parties invoke Rotation Check for each $(\mathsf{s}_\mathsf{b}{}^{(j-1)}, \mathsf{s}_\mathsf{b}{}^{(j)}, -1)$, and set $\mathsf{att\_mask} := \mathsf{a}_\mathsf{c}^{\mathsf{old}'} \cdot \left(1 - \sum_{j=1}^{k-1} \mathsf{s}_\mathsf{b}{}^{(j)}\right)$.
7. (*False negative check*) $\mathbf{P}$ and $\mathbf{V}$ invoke NoZero Check on the column with data diff and activator $\mathsf{att\_mask} \cdot (1 - \mathsf{a}_\mathsf{c}^{\mathsf{new}})$.
8. (*False positive check*)
   (a) If the pattern is *prefix* or *suffix*, $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column with data diff and activator $\mathsf{att\_mask} \cdot \mathsf{a}_\mathsf{c}^{\mathsf{new}}$.
   (b) If the pattern is *infix*, $\mathbf{P}$ sends a false-positive marker m_fp, then
      i. $\mathbf{P}$ and $\mathbf{V}$ invoke Booleanity Check on m_fp;
      ii. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on $\mathsf{m}_\mathsf{fp} \cdot (1 - \mathsf{a}_\mathsf{c}^{\mathsf{new}})$ and on $\mathsf{m}_\mathsf{fp} \cdot (1 - \mathsf{att\_mask})$.

iii. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column with data diff and activator $\mathsf{m_{fp}}$;
iv. $\mathbf{P}$ sends n_{m} to $\mathbf{V}$, then $\mathbf{P}$ and $\mathbf{V}$ invoke invoke Sumcheck twice, for $\sum_{\vec{x}} \mathsf{m_{fp}}(\vec{x}) = n_{\mathsf{m}}$ and $\sum_{\vec{x}} \mathsf{a_h^{new}}(\vec{x}) = n_{\mathsf{m}}$.
v. $\mathbf{P}$ and $\mathbf{V}$ invoke No-Duplicate Check on the column with data src and activator $\mathsf{m_{fp}}$.

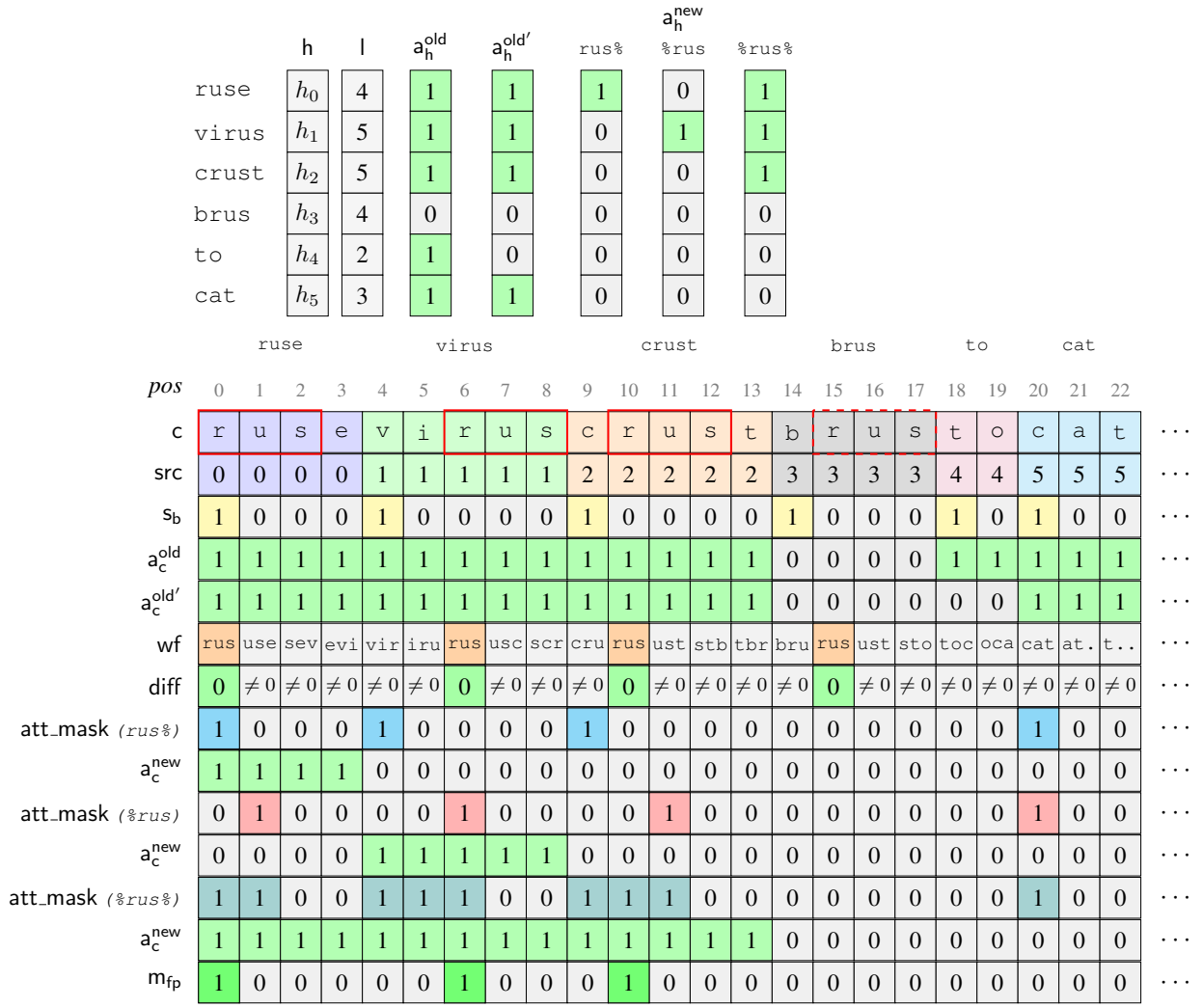

Figure 3: SFMC example for factor rus ($k = 3$)

Proof. We argue completeness and soundness, assuming each invoked sub-PIOP is complete and sound for its relation. Call a string *eligible* if it is active under $\mathsf{a_h^{old}}$ and has length at least k; by the soundness of Length Filtering Check these are exactly the strings marked by $\mathsf{a_h^{old'}}$, and we write $E := \{\, i : \mathsf{a_h^{old'}}[i] = 1 \,\}$. Let att_mask be the attention mask of the chosen filter; a string *matches* if one of its candidate windows (the positions p in it with $\mathsf{att\_mask}[p] = 1$) equals str. The filter must keep exactly the eligible matching strings, and we write $M := \{\, i : \mathsf{a_h^{new}}[i] = 1 \,\}$ for the kept set.

Completeness. For an honest prover the rotation checks pass, so $\mathsf{c}^{(\delta)}[p] = \mathsf{c}[p + \delta]$, $\mathsf{diff}[p] = 0$ exactly at the occurrences of str, and att_mask marks the candidate windows of each eligible string. The honest new

activators keep exactly the eligible matching strings, so they are boolean, consistent, and contained in $\mathsf{a}_\mathsf{h}^{\mathsf{old}'}$, passing the activator-validity and containment checks. Every dropped eligible string is non-matching, so all of its candidate windows have $\mathsf{diff} \neq 0$ and the false-negative no-zero check passes. For an anchored filter the single candidate of each kept string equals str, so the false-positive zerocheck passes; for the infix filter the prover marks with m_fp one candidate window equal to str in each kept string — boolean, satisfying $\mathsf{m}_\mathsf{fp} \leq \mathsf{a}_\mathsf{c}^{\mathsf{new}}$ and $\mathsf{m}_\mathsf{fp} \leq \mathsf{att\_mask}$, with $\mathsf{diff} = 0$ on its support, one mark per kept string, matching the announced count $n_\mathsf{m} = |M|$ — so the false-positive checks pass. Hence the honest transcript is accepted.

Soundness. Suppose $\mathbf{V}$ accepts. By the soundness of Length Filtering Check (Section 4.4), $E = \{\, i : \mathsf{a}_\mathsf{h}^{\mathsf{old}'}[i] = 1 \,\}$ is exactly the set of eligible strings and $\mathsf{a}_\mathsf{c}^{\mathsf{old}'}$ is on exactly at their characters.

The rotation checks force $\mathsf{c}^{(\delta)}[p] = \mathsf{c}[p+\delta]$ for every δ, so $\mathsf{wf}[p] = \sum_\delta r_\delta\, \mathsf{c}[p+\delta]$ is the fingerprint of the length-k window of c at p and $\mathsf{diff}[p] = \mathsf{wf}[p] - \mathsf{pf}$. A window equal to str gives $\mathsf{diff}[p] = 0$ identically; a window differing from str gives $\mathsf{diff}[p] = 0$ only when the verifier's $r_0, \ldots, r_{k-1}$ are a root of a fixed nonzero degree-1 polynomial, so by Schwartz–Zippel and a union bound over the N positions, except with probability $N(k-1)/|\mathbb{F}|$,

$$\mathsf{diff}[p] = 0 \iff \text{the length-}k \text{ window of } \mathsf{c} \text{ at } p \text{ equals } \mathsf{str}, \qquad \text{for every } p.$$

We condition on this event; note the direction $\mathsf{diff}[p] \neq 0 \Rightarrow \text{window} \neq \mathsf{str}$ holds *unconditionally*. The rotation checks on the boundary selectors make $\mathsf{att\_mask}$ the attention mask of the chosen filter: within each eligible string it marks the single anchored window (prefix, suffix) or all $\mathsf{l} - k + 1$ valid starts (infix), with $\mathsf{att\_mask} \leq \mathsf{a}_\mathsf{c}^{\mathsf{old}'}$, and in every case $\mathsf{att\_mask}[p] = 1$ guarantees that the window at p lies entirely within the string $\mathsf{src}[p]$.

The activator-validity check makes $\mathsf{a}_\mathsf{h}^{\mathsf{new}}, \mathsf{a}_\mathsf{c}^{\mathsf{new}}$ boolean and forces $\mathsf{a}_\mathsf{c}^{\mathsf{new}}[j] = \mathsf{a}_\mathsf{h}^{\mathsf{new}}[\mathsf{src}[j]]$ for every character j, so $\mathsf{a}_\mathsf{c}^{\mathsf{new}}$ is constant on each string and equals its header bit. The containment zerocheck $\mathsf{a}_\mathsf{h}^{\mathsf{new}}(1 - \mathsf{a}_\mathsf{h}^{\mathsf{old}'}) = 0$ forces $M \subseteq E$.

No false positives. For an anchored filter, $\mathsf{att\_mask}$ marks one candidate per eligible string, and the false-positive zerocheck forces $\mathsf{diff} = 0$ wherever $\mathsf{att\_mask} \cdot \mathsf{a}_\mathsf{c}^{\mathsf{new}} = 1$, i.e. at the candidate of each kept string; since that window lies within its string and $\mathsf{diff} = 0$ means it equals str, every kept string matches. For the infix filter, the two confinement zerochecks give $\mathsf{m}_\mathsf{fp} \leq \mathsf{a}_\mathsf{c}^{\mathsf{new}}$ and $\mathsf{m}_\mathsf{fp} \leq \mathsf{att\_mask}$, and the occurrence zerocheck gives $\mathsf{diff} = 0$ on the support of m_fp; thus every mark p has $\mathsf{a}_\mathsf{c}^{\mathsf{new}}[p] = 1$ (so $\mathsf{src}[p] \in M$), $\mathsf{att\_mask}[p] = 1$ (a candidate window lying inside $\mathsf{src}[p]$), and $\mathsf{diff}[p] = 0$ (that window equals str), so $\mathsf{src}[p]$ matches. The no-duplicate check on $(\mathsf{src}, \mathsf{m}_\mathsf{fp})$ puts the marks in distinct strings, and the two sum checks give $\sum \mathsf{m}_\mathsf{fp} = n_\mathsf{m} = \sum \mathsf{a}_\mathsf{h}^{\mathsf{new}} = |M|$, so the $|M|$ marks lie in distinct strings of M and hit every string of M exactly once. Either way, every kept string is an eligible match.

No false negatives. Let $i \in E \setminus M$ be a dropped eligible string. By consistency every character of i has $\mathsf{a}_\mathsf{c}^{\mathsf{new}} = 0$, so $\mathsf{att\_mask} \cdot (1 - \mathsf{a}_\mathsf{c}^{\mathsf{new}}) = \mathsf{att\_mask}$ on string i, which is 1 at each of its candidate windows. The false-negative no-zero check therefore forces $\mathsf{diff}[p] \neq 0$ there, so by the unconditional direction none of i's candidate windows equals str. Hence i does not match. Contrapositively, every eligible matching string lies in M.

Combining the two directions, M is exactly the set of eligible matching strings — the strings passing the chosen prefix, suffix, or infix filter — except with the fingerprint-collision probability $N(k-1)/|\mathbb{F}|$ and the soundness errors of the invoked sub-PIOPs. □

5.1.2 Multi-Factor

A multi-factor multi-character pattern matches several literal factors against each value, with a wild-card % between consecutive factors absorbing an arbitrary run of characters. Writing the pattern as $p_1 \% p_2 \% \ldots \% p_t$,

a value satisfies it iff the factors $\mathcal{P} = \{p_1, \ldots, p_t\}$ all occur within it *in this order*.

The objective of the protocol is to establish that the updated activators $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$ and $\mathsf{a}_{\mathsf{c}}^{\mathsf{new}}$ correctly capture the result of the pattern matching. As in SFMC Pattern Matching, the first steps check the validity of the provided activators (Step 1), filter by length (Step 2), and enforce containment (Step 3).

The prover and the verifier then run SFMC Pattern Matching for each factor $p \in \mathcal{P}$, obtaining a separate activator $\mathsf{a}_{\mathsf{h},\mathsf{p}}^{\mathsf{new}}$ per factor. It is important to note that a string satisfying every factor predicate *separately* need not satisfy the full pattern $\mathcal{P}$: in general $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}} \neq \mathsf{a}_{\mathsf{h}}^{\cap}$, where $\mathsf{a}_{\mathsf{h}}^{\cap} := \prod_{p \in \mathcal{P}} \mathsf{a}_{\mathsf{h},\mathsf{p}}^{\mathsf{new}}$. The reason is that the order of the factors matters. For the pattern `%ab%cd%`, for instance, the value `xabycdz` satisfies the predicate while `xcdyabz` does not—even though both contain `ab` and `cd`—because there `cd` precedes `ab`. Hence the finally-active words form a subset of those active in every factor, which is enforced in Step 5. The rest of the protocol establishes that the words deactivated between $\mathsf{a}_{\mathsf{h}}^{\cap}$ and $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$ fail to admit the correct ordering (there are no false negatives), while the words that survive into $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$ do satisfy the ordering constraints (there are no false positives).

Ordering False-Positive Check. The easier direction is to verify that the words surviving into the final activator—those active in both $\mathsf{a}_{\mathsf{h}}^{\cap}$ and $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$—do respect the required ordering. For this the verifier needs a handle on where each factor sits inside its word: at Step 6a the prover sends, for every factor $p_i \in \mathcal{P}$, the word-level offset column offset_{p_i} recording the position at which p_i is placed. Then, at Step 6c, for each consecutive pair (p_i, p_{i+1}) the verifier checks that $\mathsf{offset}_{p_{i+1}}$ exceeds offset_{p_i} by at least $|p_i|$, so the factors appear in order and without overlap. The remaining checks ensure that each offset_{p_i} is itself well-formed—that it genuinely marks an occurrence of p inside the word.

Ordering False-Negative Check. The trickier direction is to establish that every discarded string—one that contains all the factors (active in $\mathsf{a}_{\mathsf{h}}^{\cap}$) but is switched off in the output (inactive in $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$)—genuinely fails to admit the factors in the required order. The difficulty is that a cheating prover has many ways to discard a perfectly valid ordering simply by reporting the factor placements incorrectly. Consider the pattern `%ab%bd%bd%` ($\mathcal{P} = \{p_1, p_2, p_3\} = \{\mathtt{ab}, \mathtt{bd}, \mathtt{bd}\}$), in which the factor `bd` repeats, together with the string `xabybdzbd`: it matches the pattern (`ab` at 1, `bd` at 4, `bd` at 7) and should be active in the output. A cheating prover, however, could report the placements of p_2 and p_3 as 7 and 4—swapping the two `bd` occurrences—so that the offsets appear out of order and the string is wrongly discarded.

To rule out this attack, the verifier forces offset_{p_i} to be formed greedily: each factor is placed at the earliest match at or past the end of the previous factor, never skipping a valid occurrence. This greedy placement is optimal—if any in-order embedding exists, the greedy one does too—so once the verifier is convinced that even the greedy offsets cannot place all factors in order, no placement can, and the string is rightly discarded.

To enforce greedy offsetting, the core idea is to keep track of three things: (i) *all* occurrences of the factors, which is denoted by $\mathsf{occurs}_{\mathsf{p}}$ masking polynomial. (ii) The set of allowed offsets that a factor can get, which starts when the previous factor is completely finished, i.e. $\mathsf{offset}p_{j-1} + |p_{j-1}|$, which is denoted by $\mathsf{allowed\_offset}$ masking polynomial. (iii) The set of offsets that could have been used for the factor p_j but was not used, which is denoted by $\mathsf{unused\_offset}$ masking polynomial. Now, the greediness of the offsets can be established by a zerocheck on the multiplication on these three polynomials which means: It's impossible for a factor p_j to occur in an allowed offset which is smaller than the recorded offset. This is checked in Step 7b.

PIOP 6: Multi-Factor Multi-Character (MFMC) Pattern Matching

1. (*Activator validity check*)
   (a) $\mathbf{P}$ and $\mathbf{V}$ invoke Booleanity Check on $\mathsf{a}_{\mathsf{c}}^{\mathsf{new}}$ and on $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$.
   (b) $\mathbf{P}$ and $\mathbf{V}$ invoke Activator Consistency Check on $(\mathsf{a}_{\mathsf{c}}^{\mathsf{new}}, \mathsf{a}_{\mathsf{h}}^{\mathsf{new}})$.

2. (*Filtering by length*) $\mathbf{P}$ computes $\mathsf{a}_{\mathsf{h}}^{\mathsf{old}'}$ and $\mathsf{a}_{\mathsf{c}}^{\mathsf{old}'}$, the result of filtering out strings shorter than $k := \sum_{p\in\mathcal{P}} |p|$ (the least length a string can have while containing all factors without overlap), then $\mathbf{P}$ and $\mathbf{V}$ invoke Length Filtering Check $(\mathsf{a}_{\mathsf{h}}^{\mathsf{old}}, \mathsf{a}_{\mathsf{c}}^{\mathsf{old}}, \mathsf{a}_{\mathsf{h}}^{\mathsf{old}'}, \mathsf{a}_{\mathsf{c}}^{\mathsf{old}'})$ with threshold k.
3. (*Containment in the input*) $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}} \cdot (1 - \mathsf{a}_{\mathsf{h}}^{\mathsf{old}'})$.
4. (*Checking patterns separately*) For each *distinct* $p \in \mathcal{P}$:
   (a) $\mathbf{P}$ sends $\mathsf{a}_{\mathsf{h,p}}^{\mathsf{new}}$ and $\mathsf{a}_{\mathsf{c,p}}^{\mathsf{new}}$ to $\mathbf{V}$, and the parties invoke SFMC Pattern Matching over p. This run forms the window-fingerprint difference diff_p with an attention mask $\mathsf{att\_mask}_p$.
   (b) $\mathbf{P}$ sends the *full occurrence column* occurs_p, which is 1 at the start of *every* occurrence of p. $\mathbf{P}$ and $\mathbf{V}$ then perform the following steps to establish the validity of occurs_p:
      i. a Booleanity Check on occurs_p;
      ii. a Zerocheck on $\mathsf{occurs}_p \cdot (1 - \mathsf{att\_mask}_p)$;
      iii. a Zerocheck on $\mathsf{occurs}_p \cdot \mathsf{diff}_p$;
      iv. a NoZero Check on the column with data diff_p and activator $\mathsf{att\_mask}_p \cdot (1 - \mathsf{occurs}_p)$.
5. (*Containment in the per-factor match intersection*) $\mathbf{P}$ and $\mathbf{V}$ define the per-factor match intersection $\mathsf{a}_{\mathsf{h}}^{\cap} := \prod_{p\in\mathcal{P}} \mathsf{a}_{\mathsf{h,p}}^{\mathsf{new}}$ and invoke a Zerocheck on $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}} \cdot \left(1 - \mathsf{a}_{\mathsf{h}}^{\cap}\right)$.
6. (*False-positive check for factor ordering*) $\mathbf{P}$ and $\mathbf{V}$ proceed as follows:
   (a) *(word-level offset)* For each $j = 1, \ldots, t$, $\mathbf{P}$ computes a word-level column offset_{p_j} of size N giving the start at which p_j is greedily placed in each word. With frontier $f := \mathsf{offset}_{p_{j-1}}[i] + |p_{j-1}|$ left by the previous factor ($f := 0$ when $j = 1$),

$$\mathsf{offset}_{p_j}[i] := \begin{cases} 0, & p_j \text{ does not occur in } w_i, \\ \min\{\, s \geq f : p_j \text{ occurs at } s \,\}, & p_j \text{ occurs at some } s \geq f, \\ f - |p_j|, & \text{otherwise } (p_j \text{ is stuck}). \end{cases}$$

   $\mathbf{P}$ sends offset_{p_j} to $\mathbf{V}$.
   (b) *(character-level offset marker)* For each $j = 1, \ldots, t$, $\mathbf{P}$ sends a character-level boolean column $\mathsf{offset\_marker}_{p_j}$ (of size M) marking, in each matched word where p_j is placed in order, the character at the placed start: for a character c in word $i := \mathsf{src}[c]$,

$$\mathsf{offset\_marker}_{p_j}[c] := \begin{cases} 1, & \mathsf{a}_{\mathsf{h}}^{\cap}[i] = 1,\ \mathsf{is\_ordered}_{p_j}[i] = 1,\ \text{and } \mathsf{o}[c] = \mathsf{offset}_{p_j}[i], \\ 0, & \text{otherwise.} \end{cases}$$

   $\mathbf{P}$ and $\mathbf{V}$ then invoke the following to establish its validity:
      i. a Booleanity Check on $\mathsf{offset\_marker}_{p_j}$;
      ii. a Zerocheck on $\mathsf{offset\_marker}_{p_j}(1 - \mathsf{occurs}_{p_j})$;
      iii. a Zerocheck on $\mathsf{offset\_marker}_{p_j}(1 - \mathsf{a}_{\mathsf{c}}^{\cap})$;
      iv. a No-Duplicate Check on src with activator $\mathsf{offset\_marker}_{p_j}$;
      v. two Sumchecks for $\sum_c \mathsf{offset\_marker}_{p_j}[c] = n$ and $\sum_i \mathsf{a}_{\mathsf{h}}^{\cap}[i]\, \mathsf{is\_ordered}_{p_j}[i] = n$, where n is a count sent by $\mathbf{P}$.
   (c) (*ordering marker*) For each $j = 2, \ldots, t$, $\mathbf{P}$ commits a word-level boolean column $\mathsf{is\_ordered}_{p_j}$ (of size N) defined by

$$\mathsf{is\_ordered}_{p_j}[i] := \begin{cases} 1 & \mathsf{offset}_{p_j}[i] \geq \mathsf{offset}_{p_{j-1}}[i] + |p_{j-1}| \text{ and } \mathsf{a}_{\mathsf{h}}^{\cap}[i] = 1, \\ 0 & \text{otherwise,} \end{cases}$$

   with the convention $\mathsf{is\_ordered}_{p_1} := \mathbf{1}$. $\mathbf{P}$ and $\mathbf{V}$ then invoke
   - a Booleanity Check on $\mathsf{is\_ordered}_{p_j}$;
   - a NonNeg Sign Check on $\mathsf{offset}_{p_j} - \mathsf{offset}_{p_{j-1}} - |p_{j-1}|$ with activator $\mathsf{a}_{\mathsf{h}}^{\cap}\, \mathsf{is\_ordered}_{p_j}$;

   (d) *(consistency between the offset and offset marker)* $\mathbf{V}$ then samples $\{r_j\}_{j=1}^{t} \xleftarrow{\$} \mathbb{F}$ and sends them to $\mathbf{P}$, then

they both run Multiset Equality Check on $\bigcup_{j=1}^{t}\{\,\mathsf{ind}[i]+r_j\,\mathsf{offset}_{p_j}[i]\;:\;\mathsf{a}_{\mathsf{h}}^{\cap}[i]=1 \text{ and } \mathsf{is\_ordered}_{p_j}[i]=1\,\}$ and $\bigcup_{j=1}^{t}\{\,\mathsf{src}[c]+r_j\,\mathsf{o}[c]\;:\;\mathsf{offset\_marker}_{p_j}[c]=1\,\}$.

(e) *(verdict)* For each $j=2,\ldots,t$, the parties invoke a Zerocheck on $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}\,(1-\mathsf{is\_ordered}_{p_j})$.

7. (*False-negative check for factor ordering*) $\mathbf{P}$ and $\mathbf{V}$ denote by $\mathsf{d_h} := \mathsf{a}_{\mathsf{h}}^{\cap} - \mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$ the *discarded candidates*.

(a) (*broadcasting*) $\mathbf{P}$ sends the columns $\mathsf{d_h}'$, offset'_{p_j} for $j=1,\ldots,t$ (with $\mathsf{offset}'_{p_0}:=0$), and $\mathsf{is\_ordered}'_{p_j}$ for $j=2,\ldots,t$ (with $\mathsf{is\_ordered}'_{p_1}:=\mathbf{1}$), where x' denotes the broadcasted version of x. Then $\mathbf{P}$ and $\mathbf{V}$ invoke Broadcast Check on all of the broadcasted polynomials.

(b) (*the offset is greedy*) For each $j=1,\ldots,t$ the parties invoke:

- a NonNeg Sign Check on $\mathsf{o}-\big(\mathsf{offset}'_{p_{j-1}}+|p_{j-1}|\big)$, giving a boolean column $\mathsf{allowed\_offset}_{p_j}$ marking $\mathsf{o}\geq\mathsf{offset}'_{p_{j-1}}+|p_{j-1}|$, with activator $\mathsf{d_h}'$ (so $\mathsf{allowed\_offset}_{p_j}$ flags the offsets from which p_j may still start—those at or past the end of the previously placed factor);
- a Neg Sign Check on $\mathsf{o}-\mathsf{offset}'_{p_j}$, giving a boolean column $\mathsf{unused\_offset}_{p_j}$ marking $\mathsf{o}<\mathsf{offset}'_{p_j}$, with activator $\mathsf{d_h}'$ (so $\mathsf{unused\_offset}_{p_j}$ flags the offsets strictly before the placement claimed for p_j; together, $\mathsf{allowed\_offset}_{p_j}\cdot\mathsf{unused\_offset}_{p_j}$ picks out exactly the window $\mathsf{offset}'_{p_{j-1}}+|p_{j-1}|\leq\mathsf{o}<\mathsf{offset}'_{p_j}$);
- Zerocheck on $\mathsf{occurs}_{p_j}\cdot\mathsf{allowed\_offset}_{p_j}$, with activator $\mathsf{d_h}'(1-\mathsf{is\_ordered}'_{p_j})$ (it is impossible for p_j to occur at any offset o with $\mathsf{o}\geq\mathsf{offset}'_{p_{j-1}}+|p_{j-1}|$, so the scan genuinely has *no* occurrence past the previous factor and is stuck).
- *(verdict)* Zerocheck on $\mathsf{occurs}_{p_j}\cdot\mathsf{allowed\_offset}_{p_j}\cdot\mathsf{unused\_offset}_{p_j}$, with activator $\mathsf{d_h}'\,\mathsf{is\_ordered}'_{p_j}$ (it is impossible for p_j to occur at an offset o with $\mathsf{offset}'_{p_{j-1}}+|p_{j-1}|\leq\mathsf{o}<\mathsf{offset}'_{p_j}$, so offset'_{p_j} is the *earliest* occurrence past the previous factor);

(c) (*verdict*) The parties invoke Zerocheck on $\mathsf{d_h}\cdot\prod_{j=1}^{t}\mathsf{is\_ordered}_{p_j}$.

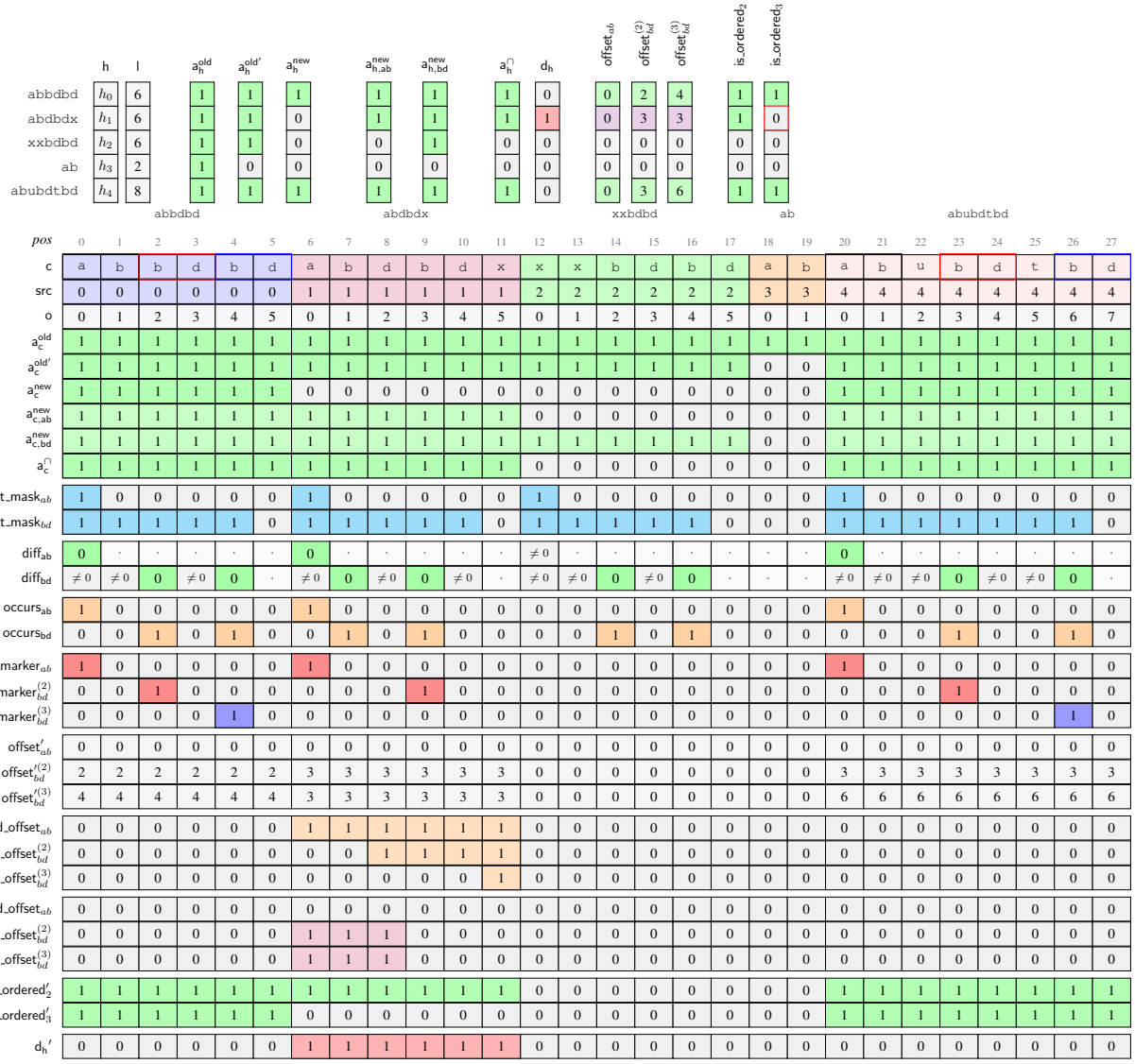

Figure 4: TODO

6 Fingerprinting

A string fingerprint can be understood as the answer vector to a small collection of yes/no questions asked of a string. In the most precise form one could imagine, there would be one such question per *atomic feature* of interest—for instance, "does this string contain the letter `a`?", "does it contain the bigram `th`?", "does it start with a digit?", or any other Boolean property one cares to evaluate. Evaluating every atomic feature gives a lossless, but potentially very large, summary of the string.

The fingerprint trades this precision for size. Instead of storing one bit per atomic feature, we partition the features into n groups and *or* them together: a single bit fires whenever *any* of its constituent features fires. Each bit of the fingerprint is therefore not a single feature but the disjunction of many features. This compression is what makes fingerprints small and fast to evaluate—and also what makes them lossy. The design problem is then twofold: choose the family of atomic features to draw from, and choose how to group them into bits.

Let $\mathcal{F} = \{f_1, f_2, \ldots, f_m\}$ be a finite family of *atomic features*, where each f_i is a Boolean function on strings:

$$f_i : \Sigma^* \to \{0, 1\}.$$

We make no assumption about what each f_i computes; it may test for the presence of a character, the presence of an n-gram, a positional condition, the output of a hash function, or any other property.

A fingerprint of bitwidth n is induced by a partition $\{B_1, \ldots, B_n\}$ of $\mathcal{F}$. The j-th bit of the fingerprint of a string s is the disjunction of the features assigned to bin B_j:

$$\mathbf{f}_s[j] \;=\; \bigvee_{f \in B_j} f(s). \tag{1}$$

The full fingerprint $\mathbf{f}_s \in \{0, 1\}^n$ is the vector of these disjunctions.

Example. Let $\Sigma = \{a, e, h, l, n, t, u\}$ and take the family $\mathcal{F} = \{f_a, f_e, f_h, f_l, f_n, f_t, f_u, g_{\text{th}}, g_{\text{ut}}\}$, where $f_c(s) = \mathbb{1}[c \in s]$ and $g_{xy}(s) = \mathbb{1}[xy \text{ occurs in } s]$. With bitwidth $n = 3$ and partition $B_1 = \{f_a, f_l, f_u\}$, $B_2 = \{f_e, f_h, f_n, f_t\}$, $B_3 = \{g_{\text{th}}, g_{\text{ut}}\}$, the fingerprints of a few strings are:

String	$\mathbf{f}[1]$	$\mathbf{f}[2]$	$\mathbf{f}[3]$
`utn`	1	1	1
`lute`	1	1	1
`heat`	1	1	0
`ten`	0	1	0

For instance, `lute` sets $\mathbf{f}[3] = 1$ via the bigram `ut`; the fingerprint records that *some* feature in B_3 fired, but not which.

6.1 Approximate Filtering

Fingerprints become useful when we want to quickly decide whether a candidate string s could possibly contain a pattern ζ, without performing the full containment test.

The pruning rule. For a pattern ζ and a candidate string s, the fingerprint check accepts s iff

$$\mathbf{f}_\zeta \subseteq \mathbf{f}_s \iff \forall j \in [n] : \mathbf{f}_\zeta[j] = 1 \Rightarrow \mathbf{f}_s[j] = 1. \tag{2}$$

For this check to be a sound filter—never rejecting a true match—the feature family $\mathcal{F}$ must satisfy a *monotonicity* condition: whenever ζ is contained in s in the sense the query intends, every feature satisfied by ζ must also be satisfied by s,

$$\zeta \sqsubseteq s \implies \forall f \in \mathcal{F} : f(\zeta) \leq f(s). \tag{3}$$

Under (3), the subset check (2) cannot produce false negatives.

Where false positives come from. The converse of the subset check fails because of the *or*-compression in (1). A string s may light bit j via a feature $f \in B_j$ unrelated to those the pattern requires; the fingerprint cannot tell which feature of B_j caused the bit to fire. Two design choices govern how often this happens:

1. The **feature family** $\mathcal{F}$. Richer families (e.g., n-grams, positional features) carry more information per atomic feature and therefore tolerate coarser bin assignments, at the cost of a larger $|\mathcal{F}|$ and a more expensive partition-selection problem.

2. The **partition** $\{B_1, \ldots, B_n\}$. Grouping features the workload needs to *distinguish* into the same bin destroys that information; grouping interchangeable features wastes little.

Example. Reusing the setup from above, take pattern $\zeta =$ `utn` with $\mathbf{f}_\zeta = (1, 1, 1)$. Then `lute` passes the filter as a false positive (its $\mathbf{f}[3]$ bit is lit by `ut` rather than by `utn`), while `heat` is correctly rejected on $\mathbf{f}[3]$ and `ten` on $\mathbf{f}[1]$. Replacing B_3 with another character bin would weaken the filter on `heat`; merging `u` and `t` into one bin would weaken it on `ten`.

Now building on this tool, we design a PIOP where

PIOP 7: Approximate Filtering Check

1. $\mathbf{P}$ and $\mathbf{V}$ compute the fingerprint of the pattern ζ as $\mathbf{f}_\zeta$ and the field element encoding of the fingerprint as $f_\zeta = \sum_{j=0}^{n-1} 2^j \mathbf{f}_\zeta[j]$.
2. $\mathbf{P}$ divides the fingerprint col

7 Optimizations

7.1 Prefix and Suffix Check

The substring protocol of SFMC Pattern Matching already handles prefix and suffix as its single-candidate case, but the anchored filters admit a cheaper specialized protocol: under our MSM-based commitment it commits only binary boundary selectors, avoiding the $k - 1$ non-binary rotations of c that the substring protocol sends. We present it here. A prefix/suffix check keeps the strings in a column that begin or end with a fixed pattern str of length k: a *prefix* filter $\mathsf{str}\%$ keeps strings that start with str (as in `LIKE 'str%'`), and a *suffix* filter $\%\mathsf{str}$ keeps strings that end with it (as in `LIKE '%str'`).

What the protocol proves. The verifier is given a tuple $(\mathsf{h}, \mathsf{l}, \mathsf{a}_\mathsf{h}^{\mathsf{old}}, \mathsf{c}, \mathsf{src}, \mathsf{a}_\mathsf{c}^{\mathsf{old}}, \mathsf{s}_\mathsf{b})$ and new activators $\mathsf{a}_\mathsf{h}^{\mathsf{new}}$ and $\mathsf{a}_\mathsf{c}^{\mathsf{new}}$. The prover convinces the verifier that the tuple $(\mathsf{h}, \mathsf{l}, \mathsf{a}_\mathsf{h}^{\mathsf{new}}, \mathsf{c}, \mathsf{src}, \mathsf{a}_\mathsf{c}^{\mathsf{new}}, \mathsf{s}_\mathsf{b})$ is the result of applying the filter to the original tuple.

At a high level the check has several parts, which we explain in turn before presenting the protocol. For concreteness we describe the prefix scenario throughout; afterwards we show how the prefix and suffix protocols can be unified into a single check.

Activator consistency. The prover claims that $\mathsf{a}_\mathsf{h}^{\mathsf{new}}$ and $\mathsf{a}_\mathsf{c}^{\mathsf{new}}$ are the activators of the filtered column. The verifier checks that they are internally consistent with one another and with the lengths: a string is marked active by $\mathsf{a}_\mathsf{h}^{\mathsf{new}}$ exactly when its characters are marked active by $\mathsf{a}_\mathsf{c}^{\mathsf{new}}$, and the active-character count of each string equals its length.

Filtering by length. A string shorter than k can never match a length-k pattern, and leaving it in place is actively harmful: once the anchored selectors are rotated inward, a short string's selectors run past its own end and spill into the character slots of its neighbors, corrupting the comparison. The first step therefore removes every string of length less than k by invoking the Length Filtering Check (Section 4.4), which returns masked activators $\mathsf{a}_\mathsf{h}^{\mathsf{old}'}$ and $\mathsf{a}_\mathsf{c}^{\mathsf{old}'}$ equal to $\mathsf{a}_\mathsf{h}^{\mathsf{old}}, \mathsf{a}_\mathsf{c}^{\mathsf{old}}$ with every short string (and all of its characters) switched off. All later steps use these in place of the originals, so that they range only over strings long enough to be candidate matches.

Containment. A string can only survive the filter if it was eligible to begin with. A Zerocheck on $\mathsf{a}_\mathsf{h}^{\mathsf{new}} \cdot (1 - \mathsf{a}_\mathsf{h}^{\mathsf{old}'})$ forces $\mathsf{a}_\mathsf{h}^{\mathsf{new}} \le \mathsf{a}_\mathsf{h}^{\mathsf{old}'}$, so the prover cannot retain a short or inactive string — one whose anchored slots, and hence the pattern comparison below, are switched off. Every kept string is therefore eligible, with all k of its anchored slots live.

False positives Check. A string that *survives* the filter must genuinely match the pattern. The pattern column p already holds the required character at every anchored slot, so this reduces to a single check that the data agrees with p there. The parties run a Zerocheck on $\mathsf{c} - \mathsf{p}$ with activator $\mathsf{a}_\mathsf{c}^{\mathsf{new}} \cdot \sum_{i=0}^{k-1} \mathsf{s}_\mathsf{b}'^{(i)}$, which is on exactly at the anchored slots of the kept strings; the check therefore forces $\mathsf{c} = \mathsf{p}$ at every one of them, so no kept string disagrees with the pattern at any position.

False negatives Check. Conversely, a string that the prover *drops* must genuinely fail the pattern; otherwise a real match would be silently discarded. A single mismatched position is enough to rule a string out, so rather than re-prove the whole comparison, the prover exhibits one witness per dropped string. It sends a boolean *mismatch selector* s_n that places a single 1 inside each *dropped candidate* — a string that was eligible ($\mathsf{a}_\mathsf{c}^{\mathsf{old}'} = 1$: active and at least k long) but not kept ($\mathsf{a}_\mathsf{c}^{\mathsf{new}} = 0$) — at an anchored slot where its character violates the pattern, and is 0 everywhere else. For this to rule out false negatives, s_n must be *well-formed*: it must place exactly one mark in each non-matching eligible string, and that mark must sit at a genuine mismatch. We establish this with a counting argument. A Booleanity Check forces s_n to be $\{0,1\}$-valued, a No-Duplicate Check on the source indices carried by the marked slots forces those slots to lie in distinct strings, so each string holds at most one mark, and a Zerocheck on $\mathsf{s}_\mathsf{n} \cdot (1 - \sum_{i=0}^{k-1} \mathsf{s}_\mathsf{b}'^{(i)})$ confines every mark to an anchored slot of an eligible string (so a mark cannot be parked on an inactive slot where $\mathsf{p} = 0$ would make the mismatch test pass vacuously). The prover then announces the number of matches n_m and non-matches n_nm, and three sumchecks pin these down: $\sum \mathsf{a}_\mathsf{h}^{\mathsf{new}} = n_\mathsf{m}$ counts the kept strings, $\sum \mathsf{s}_\mathsf{n} = n_\mathsf{nm}$ counts the marks, and $\sum \mathsf{a}_\mathsf{h}^{\mathsf{old}'} = n_\mathsf{m} + n_\mathsf{nm}$ counts the eligible strings. Since every eligible string is either kept or marked but not both (a marked slot has $\mathsf{c} \neq \mathsf{p}$, which a kept string never has), the identity $n_\mathsf{m} + n_\mathsf{nm} = \sum \mathsf{a}_\mathsf{h}^{\mathsf{old}'}$ forces the marks to cover *every* non-matching eligible string. Finally, a NoZero Check on $\mathsf{c} - \mathsf{p}$ with activator s_n confirms every marked slot is a genuine mismatch ($\mathsf{c} \neq \mathsf{p}$ there), so each dropped string really fails the pattern.

Anchored selectors and the pattern column. Let $\rho_t(\cdot)$ denote rotation of a column by t positions (a 1 at index p moves to index $p + t$). The boundary selector s_b from the tuple marks each string's first character. Once the masked activator $\mathsf{a}_\mathsf{c}^{\mathsf{old}'}$ is in hand (it switches off short and inactive strings; see below), the parties

form the *masked boundary selector*

$$\mathsf{s}'_{\mathsf{b}}{}^{(0)} := \mathsf{a}_{\mathsf{c}}^{\mathsf{old}'} \cdot \mathsf{s}_{\mathsf{b}},$$

which keeps a boundary mark only on the strings that survive into the comparison. The prover then sends the further *anchored selectors* $\mathsf{s}'_{\mathsf{b}}{}^{(1)}, \ldots, \mathsf{s}'_{\mathsf{b}}{}^{(k-1)}$, defined by

$$\mathsf{s}'_{\mathsf{b}}{}^{(i)} := \rho_i\big(\mathsf{s}'_{\mathsf{b}}{}^{(0)}\big), \qquad i = 0, \ldots, k-1,$$

so that $\mathsf{s}'_{\mathsf{b}}{}^{(i)}$ marks the $(i{+}1)$-th character of every surviving string. Each $\mathsf{s}'_{\mathsf{b}}{}^{(i)}$ is tied to $\mathsf{s}'_{\mathsf{b}}{}^{(0)}$ by a Rotation Check. Because the short strings were removed *before* rotating, no selector runs off the end of a string into a neighbor, so the marks never collide. The parties then define the *pattern column*

$$\mathsf{p} := \sum_{i=0}^{k-1} \mathsf{s}'_{\mathsf{b}}{}^{(i)} \cdot \mathsf{str}[i],$$

which carries the required pattern character $\mathsf{str}[i]$ at the $(i{+}1)$-th slot of every surviving string and is 0 elsewhere — exactly the pattern the prefix filter expects laid over the anchored slots.

Unifying prefix and suffix. The two cases differ only in which end the comparison is anchored to. A suffix filter anchors at each string's *last* character, so the masked anchor is $\mathsf{s}'_{\mathsf{a}}{}^{(0)} := \mathsf{a}_{\mathsf{c}}^{\mathsf{old}'} \cdot \rho_{-1}(\mathsf{s}_{\mathsf{b}})$ and the prover sends the backward rotations $\mathsf{s}'_{\mathsf{a}}{}^{(i)} := \rho_{-i}\big(\mathsf{s}'_{\mathsf{a}}{}^{(0)}\big)$, forming the pattern column $\mathsf{p} := \sum_{i=0}^{k-1} \mathsf{s}'_{\mathsf{a}}{}^{(i)} \cdot \mathsf{str}[k-1-i]$, which places $\mathsf{str}[k-1]$ on the last character, $\mathsf{str}[k-2]$ on the second-to-last, and so on. Writing $\epsilon := +1$ for a prefix and $\epsilon := -1$ for a suffix, both cases use masked anchored selectors $\rho_{\epsilon i}(\text{anchor})$ and pattern characters $\mathsf{str}[\pi(i)]$ with $\pi(i) := \frac{(k-1)(1-\epsilon)}{2} + \epsilon\, i$, so $\pi(i) = i$ for a prefix and $k-1-i$ for a suffix.

PIOP 8: Prefix/Suffix Check

1. (*Activator validity check*)
   (a) $\mathbf{P}$ and $\mathbf{V}$ invoke Booleanity Check on $\mathsf{a}_{\mathsf{c}}^{\mathsf{new}}$ and on $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$.
   (b) $\mathbf{P}$ and $\mathbf{V}$ invoke Activator Consistency Check on $(\mathsf{src}, \mathsf{a}_{\mathsf{c}}^{\mathsf{new}}, \mathsf{l}, \mathsf{a}_{\mathsf{h}}^{\mathsf{new}})$.
2. (*Filtering by length*) $\mathbf{P}$ and $\mathbf{V}$ invoke Length Filtering Check on $(\mathsf{src}, \mathsf{l}, \mathsf{a}_{\mathsf{h}}^{\mathsf{old}}, \mathsf{a}_{\mathsf{c}}^{\mathsf{old}})$ with threshold k, obtaining the masked activators $\mathsf{a}_{\mathsf{h}}^{\mathsf{old}'}$ and $\mathsf{a}_{\mathsf{c}}^{\mathsf{old}'}$.
3. (*Containment*) $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}} \cdot (1 - \mathsf{a}_{\mathsf{h}}^{\mathsf{old}'})$, forcing every kept string to be eligible.
4. (*Pattern selectors*) $\mathbf{P}$ and $\mathbf{V}$ define $\mathsf{s}'_{\mathsf{b}}{}^{(0)} := \mathsf{a}_{\mathsf{c}}^{\mathsf{old}'} \cdot \mathsf{s}_{\mathsf{b}}$ as the first pattern selector, then
   (a) $\mathbf{P}$ sends $\mathsf{s}'_{\mathsf{b}}{}^{(1)}, \ldots, \mathsf{s}'_{\mathsf{b}}{}^{(k-1)}$, each $\mathsf{s}'_{\mathsf{b}}{}^{(i)}$ being the rotation of $\mathsf{s}'_{\mathsf{b}}{}^{(i-1)}$ by ϵ.
   (b) For $i = 1, \ldots, k-1$ the parties invoke Rotation Check on $(\mathsf{s}'_{\mathsf{b}}{}^{(i-1)}, \mathsf{s}'_{\mathsf{b}}{}^{(i)}, \epsilon)$.
   (c) $\mathbf{P}$ and $\mathbf{V}$ then define the pattern column $\mathsf{p} := \sum_{i=0}^{k-1} \mathsf{s}'_{\mathsf{b}}{}^{(i)} \cdot \mathsf{str}[\pi(i)]$.
5. (*False Positive Check*) $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $\big(\mathsf{c} - \mathsf{p},\ \ \mathsf{a}_{\mathsf{c}}^{\mathsf{new}} \cdot \sum_{i=0}^{k-1} \mathsf{s}'_{\mathsf{b}}{}^{(i)}\big)$.
6. (*False Negative Check*) $\mathbf{P}$ sends the boolean mismatch selector s_{n}.
   (a) $\mathbf{P}$ and $\mathbf{V}$ invoke NoZero Check on the column $(\mathsf{c} - \mathsf{p},\ \mathsf{s}_{\mathsf{n}})$, confirming each marked slot is a genuine mismatch.
   (b) (s_{n} *well-formedness*)
      i. $\mathbf{P}$ and $\mathbf{V}$ invoke Booleanity Check on s_{n}.
      ii. $\mathbf{P}$ and $\mathbf{V}$ invoke No-Duplicate Check on the column with data src and activator s_{n}, so no two marks fall in the same string.
      iii. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $\mathsf{s}_{\mathsf{n}} \cdot \big(1 - \sum_{i=0}^{k-1} \mathsf{s}'_{\mathsf{b}}{}^{(i)}\big)$, confining each mark to an anchored slot of an eligible string.
      iv. $\mathbf{P}$ sends the match count n_{m} and the non-match count n_{nm}.
      v. $\mathbf{P}$ and $\mathbf{V}$ invoke Sumcheck on $\mathsf{a}_{\mathsf{h}}^{\mathsf{new}}$ with claimed sum n_{m} (the number of matches).

vi. $\mathbf{P}$ and $\mathbf{V}$ invoke Sumcheck on $\mathsf{s_n}$ with claimed sum n_{nm} (the number of non-matches).
vii. $\mathbf{P}$ and $\mathbf{V}$ invoke Sumcheck on $\mathsf{a_h^{old'}}$ with claimed sum $n_{\mathsf{m}} + n_{\mathsf{nm}}$ (the number of eligible strings).

Proof. We argue completeness and soundness, assuming each invoked sub-PIOP is complete and sound for its relation. Call a string *eligible* if it is active under $\mathsf{a_h^{old}}$ and has length at least k; by construction these are exactly the strings marked by $\mathsf{a_h^{old'}}$, and we write $E := \{\, i : \mathsf{a_h^{old'}}[i] = 1 \,\}$ for this set. The filter must keep exactly the eligible strings that match str.

Completeness. By the completeness of Length Filtering Check, an honest prover's masked activators $\mathsf{a_h^{old'}}, \mathsf{a_c^{old'}}$ switch off precisely the short and inactive strings. The masked boundary selector and its rotations are genuine rotations, so the rotation checks pass and p carries $\mathsf{str}[\pi(i)]$ on the $(i{+}1)$-th anchored slot of every eligible string. The honest new activators keep exactly the eligible matching strings, so they are boolean, consistent, contained in $\mathsf{a_h^{old'}}$, and agree with p on every anchored slot of a kept string, passing the activator-validity and false-positive checks. For each eligible non-matching string the prover marks one anchored slot where $\mathsf{c} \neq \mathsf{p}$; this $\mathsf{s_n}$ is boolean, places no two marks in one string, lies on anchored slots, and the announced counts satisfy the three sum checks, so every false-negative check passes. Hence the honest transcript is accepted.

Soundness. Suppose $\mathbf{V}$ accepts. We show that the kept set $M := \{\, i : \mathsf{a_h^{new}}[i] = 1 \,\}$ is exactly the set of eligible matching strings; since a match requires length at least k and original activity, this is precisely the set of strings passing the filter.

By the soundness of Length Filtering Check (Section 4.4), the masked activators $\mathsf{a_h^{old'}}, \mathsf{a_c^{old'}}$ are on exactly for the active strings of length at least k and their characters; hence $E = \{\, i : \mathsf{a_h^{old'}}[i] = 1 \,\}$ is precisely the set of eligible strings, as claimed.

The activator-validity check makes $\mathsf{a_h^{new}}, \mathsf{a_c^{new}}$ boolean and mutually consistent: a string is on in $\mathsf{a_h^{new}}$ iff all of its characters are on in $\mathsf{a_c^{new}}$, and its active-character count equals its length. The containment zerocheck $\mathsf{a_h^{new}}(1 - \mathsf{a_h^{old'}}) = 0$ forces $M \subseteq E$, so every kept string is eligible and, by consistency, all of its characters carry $\mathsf{a_c^{new}} = 1$.

No false positives. For $i \in M$ the anchored slots of string i have $\mathsf{a_c^{new}} = 1$ and $\sum_t \mathsf{s_b'}^{(t)} = 1$, so the false-positive zerocheck forces $\mathsf{c} = \mathsf{p}$ there. As p carries $\mathsf{str}[\pi(t)]$ at the $(t{+}1)$-th anchored slot, string i matches str. Thus every kept string is an eligible match.

No false negatives. Let $N := \{\, i : \sum_{\mathsf{src}[j]=i} \mathsf{s_n}[j] = 1 \,\}$ be the marked strings. Booleanity of $\mathsf{s_n}$ and the no-duplicate check on $(\mathsf{src}, \mathsf{s_n})$ give at most one mark per string, so $\sum \mathsf{s_n} = |N|$, while the confinement zerocheck $\mathsf{s_n}(1 - \sum_t \mathsf{s_b'}^{(t)}) = 0$ places every mark on an anchored slot of an eligible string, so $N \subseteq E$. The nonzero check on $(\mathsf{c} - \mathsf{p}, \mathsf{s_n})$ makes $\mathsf{c} \neq \mathsf{p}$ at each marked slot; since that slot is anchored, $\mathsf{p} = \mathsf{str}[\pi(t)]$, so the marked string mismatches str and is non-matching. In particular $N \cap M = \emptyset$, since a kept eligible string agrees with p on all of its anchored slots.

The three sum checks give $|M| = \sum \mathsf{a_h^{new}} = n_{\mathsf{m}}$, $|N| = \sum \mathsf{s_n} = n_{\mathsf{nm}}$, and $|E| = \sum \mathsf{a_h^{old'}} = n_{\mathsf{m}} + n_{\mathsf{nm}} = |M| + |N|$. As $M, N \subseteq E$ are disjoint with sizes summing to $|E|$, they partition $E = M \sqcup N$. Every eligible string is therefore either kept or marked, and the marked ones are non-matching, so no eligible matching string is dropped. Combined with the no-false-positive direction, M is exactly the set of eligible matching strings, i.e. the strings passing the prefix filter. The suffix case is identical, with $\epsilon = -1$ and $\pi(i) = k - 1 - i$. □

7.1.1 Multi-Factor

c	h	l	m_h	a_h^{old}	$a_h^{old'}$	a_h^{new}
age	h_0	3	0	1	1	0
truth	h_1	5	0	0	0	0
trip	h_2	4	0	1	1	0
to	h_3	2	1	1	0	0
run	h_4	3	0	1	1	0
arch	h_5	4	0	1	1	0
trust	h_6	5	0	1	1	1
$\perp$	0	0	0	0	0	0

	age			truth					trip				to		run			arch				trust						
pos	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26		35
c	a	g	e	t	r	u	t	h	t	r	i	p	t	o	r	u	n	a	r	c	h	t	r	u	s	t	0	⋯	0
src	0	0	0	1	1	1	1	1	2	2	2	2	3	3	4	4	4	5	5	5	5	6	6	6	6	6	7	⋯	7
s_b	1	0	0	1	0	0	0	0	1	0	0	0	1	0	1	0	0	1	0	0	0	1	0	0	0	0	0	⋯	0
$s_b'^{(0)}$	1	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	1	0	0	0	1	0	0	0	0	0	⋯	0
$s_b'^{(1)}$	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	1	0	0	0	1	0	0	0	0	⋯	0
$s_b'^{(2)}$	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	1	0	0	0	1	0	0	0	⋯	0
p	t	r	u	0	0	0	0	0	t	r	u	0	0	0	t	r	u	t	r	u	0	t	r	u	0	0	0	⋯	0
m_c	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	⋯	0
a_c^{old}	1	1	1	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	⋯	0
$a_c^{old'}$	1	1	1	0	0	0	0	0	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	⋯	0
a_c^{new}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	0	⋯	0
s_n	1	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	⋯	0

Figure 5: Prefix instance ($\epsilon = +1$) of the check for `tru%` (str $=$ `tru`, $k = 3$). The length-2 string `to` (dotted) is too short for a 3-character prefix: the masks m_h, m_c flag it, and the masked activators $a_h^{old'} = a_h^{old}(1 - m_h)$, $a_c^{old'} = a_c^{old}(1 - m_c)$ deactivate it. `truth` matches the prefix but is inactive (dashed); `trust` (solid) is the active match.

c	h	l	m_h	a_h^{old}	$a_h^{old'}$	a_h^{new}
age	h_0	3	0	1	1	0
dust	h_1	4	0	0	0	0
trip	h_2	4	0	1	1	0
to	h_3	2	1	1	0	0
run	h_4	3	0	1	1	0
arch	h_5	4	0	1	1	0
trust	h_6	5	0	1	1	1
$\perp$	0	0	0	0	0	0

		age				dust				trip				to		run			arch					trust							
pos	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25		35
c	a	g	e	d	u	s	t	t	r	i	p	t	o	r	u	n	a	r	c	h	t	r	u	s	t	0	⋯	0
src	0	0	0	1	1	1	1	2	2	2	2	3	3	4	4	4	5	5	5	5	6	6	6	6	6	7	⋯	7
s_b	1	0	0	1	0	0	0	1	0	0	0	1	0	1	0	0	1	0	0	0	1	0	0	0	0	0	⋯	0
$s_a'^{(0)}$	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	1	0	0	0	0	1	0	⋯	0
$s_a'^{(1)}$	0	1	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	1	0	0	0	0	1	0	0	⋯	0
$s_a'^{(2)}$	1	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	1	0	0	0	0	1	0	0	0	⋯	0
p	u	s	t	0	0	0	0	0	u	s	t	0	0	u	s	t	0	u	s	t	0	0	u	s	t	0	⋯	0
m_c	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	⋯	0
a_c^{old}	1	1	1	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	⋯	0
$a_c^{old'}$	1	1	1	0	0	0	0	1	1	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	0	⋯	0
a_c^{new}	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	⋯	0
s_n	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	⋯	0

Figure 6: Suffix instance ($\epsilon = -1$) of the check for `%ust` (str $=$ `ust`, $k = 3$). The masked anchor $s_a'^{(0)} = a_c^{old'} \cdot \rho_{-1}(s_b)$ marks the *last* character of each surviving string, and $s_a'^{(1)}, s_a'^{(2)}$ step backward from it; the pattern column $p = \sum_i s_a'^{(i)}\, str[k{-}1{-}i]$ lays `t`,`s`,`u` from the end. Because inactive `dust` and the short `to` are dropped *before* rotating, the selectors never spill into a neighbor and p has no collisions. `dust` ends in `ust` but is inactive (dashed); `trust` (solid) is the active match.

Acknowledgments

A Extended Preliminaries

A.1 s-Contiguous Polynomial Identity

Here we show that the s-contiguous polynomial, defined in Section 2 as:

$$\mathcal{C}_{\mu,s}(X) \;:=\; \sum_{i=1}^{\mu} s_i(1-X_i)\prod_{j=1}^{i-1}(s_jX_j+(1-s_j)(1-X_j))$$

is 1 for the first s points in $\mathscr{H}_\mu$ and 0 elsewhere. Here, $s_i = \lfloor \frac{s}{2^{\mu-i}} \rfloor \bmod 2$ is the i-th most significant bit of s. Equivalently, $\mathcal{C}_{\mu,s}(X) = 1$ if and only if $\mathcal{I}_\mu(X) < s$, and $\mathcal{C}_{\mu,s}(X) = 0$ otherwise. Here, we will provide a short derivation of this identity.

Observe that the condition $\mathcal{I}_\mu(X) < s$ is satisfied if and only if there is some position i such that:

1. **Prefix Match:** For all indices $j < i$ (bits more significant than i), the bit X_j is identical to s_j.
2. **Current Bit Lower:** At index i, the bit X_i is strictly less than s_i. Since $X_i, s_i \in \{0,1\}$, this forces $s_i = 1$ and $X_i = 0$.

The polynomial captures this logic by summing over all possible positions i where this divergence could occur. The inner product, $\prod_{j=1}^{i-1}(s_jX_j + (1-s_j)(1-X_j))$, acts as an equality indicator for the prefix, evaluating to 1 if and only if $X_j = s_j$ for all $j < i$. The term $s_i(1-X_i)$ enforces the strict inequality at the current bit. Because the "first position of difference" is unique for any distinct pair of integers, these conditions are mutually exclusive across different i. Thus, the summation yields exactly 1 if such an i exists and 0 otherwise.

A.2 Deferred Low-Level PIOPs

Sumcheck. The idea of sumcheck is to reduce an exponential sum to a sequence of univariate sums, each involving one fewer variable. At each step, the prover sends a univariate polynomial that partially sums over the remaining variables. The verifier checks consistency between rounds by evaluating each polynomial at a random point and verifying that the sum matches the previous claim. After reducing to a single point, the verifier queries the original oracle to confirm the final value.

PIOP 9: Sumcheck

Inputs: $[\![p]\!], s$.

1. $\mathbf{P}$ sends to $\mathbf{V}$ a univariate polynomial oracle p_1 such that $p_1(X_1) = \sum_{\{x_2,\ldots,x_\mu\}\in\mathscr{H}_{\mu-1}} p(X_1, x_2, \ldots, x_\mu)$.
2. $\mathbf{V}$ checks that the degree of $p_1(x_1)$ is at most the degree of p in x_1. It computes $s_1 = p_1(0) + p_1(1)$ and checks that $s_1 = s$. If not, $\mathbf{V}$ rejects. It then picks a random $r_1 \in \mathbb{F}$ and sends it to $\mathbf{P}$.
3. For each i from 2 to μ (subsequent rounds):
   (a) $\mathbf{P}$ sends to $\mathbf{V}$ the univariate polynomial $p_i(x_i) = \sum_{\{x_{i+1},\ldots,x_\mu\}\in\mathscr{H}_{\mu-i}} p(r_1, \ldots, r_{i-1}, x_i, \ldots, x_\mu)$.
   (b) $\mathbf{V}$ checks that the degree of $p_i(x_i)$ is at most the degree of p in x_i. It computes $s_i = p_i(0) + p_i(1)$ and checks that $s_i = p_{i-1}(r_{i-1})$. If not, $\mathbf{V}$ rejects. It picks a random $r_i \in \mathbb{F}$ and sends it to $\mathbf{P}$.
4. $\mathbf{P}$ sends the value $p(r_1, \ldots, r_\mu)$ for the random values $r_1, \ldots, r_\mu$ chosen by $\mathbf{V}$.
5. $\mathbf{V}$ checks that this value is correct by querying p at $(r_1, \ldots, r_\mu)$.

Zerocheck. By the Schwartz–Zippel lemma, it suffices for the prover to convince the verifier that p is zero on a random point. This check is executed via a sumcheck over the polynomial $p(x)\mathsf{Eq}(x, r)$. Throughout this paper we use this style of PIOP composition, enabling us to reason about more complex polynomial properties while growing our toolbox.

PIOP 10: Zerocheck

Inputs: $[\![p]\!]$.

1. $\mathbf{V}$ samples a vector $r \xleftarrow{\$} \mathbb{F}^{\mu}$ and sends it to $\mathbf{P}$.
2. $\mathbf{P}$ and $\mathbf{V}$ invoke Sumcheck for the claim that $\sum_{x \in \mathscr{H}_\mu} p(x) \cdot \mathsf{Eq}(x, r) = 0$.

Rational Sumcheck. The *rational sumcheck* takes two columns p, q and a claimed sum s, and proves that $\sum_{x \in \mathscr{H}_\mu} p(x)/q(x) = s$ under the promise that $q(x) \neq 0$ for all $x \in \mathscr{H}_\mu$. We package this as the relation

$$\mathscr{R}_{\mathsf{RatSum}} = \left\{ \mathtt{x} = (\mu, s),\ \mathtt{y} = ([\![\tilde{p}]\!], [\![\tilde{q}]\!]),\ \mathtt{w} = (p, q) \;\middle|\; \sum_{x \in \mathscr{H}_\mu} \frac{p(x)}{q(x)} = s \right\}.$$

The prover constructs a polynomial $h(x)$ that equals $p(x)/q(x)$ on $\mathscr{H}_\mu$ and sends its oracle to the verifier, who can then be convinced that h behaves like the intended rational function via a Zerocheck. With a verified h in hand, the parties reduce the problem to a standard Sumcheck over h.

PIOP 11: RatSumcheck

Inputs: $[\![p]\!], [\![q]\!], s$.

1. $\mathbf{P}$ constructs the polynomial h such that for all $x \in \mathscr{H}_\mu$, $h(x) = \frac{p(x)}{q(x)}$, and sends its oracle $[\![h]\!]$ to $\mathbf{V}$.
2. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck for the claim that $\forall x \in \mathscr{H}_\mu$, $h(x)\, q(x) - p(x) = 0$.
3. $\mathbf{P}$ and $\mathbf{V}$ invoke Sumcheck for the claim $\sum_{x \in \mathscr{H}_\mu} h(x) = s$.

B Deferred PIOPs

This appendix collects the low-level PIOPs and relations used throughout TruthTable++. These are standard or lightly adapted from prior work, and are deferred here so that the main text can focus on the string-filtering protocols.

B.1 NoZero Check

The *NoZero Check* takes a column $\mathsf{c} = (\mathsf{d}, \mathsf{a})$ and proves that every active entry of c is nonzero. We package this as the relation

$$\mathscr{R}_{\mathsf{NoZero}} = \{\, \mathtt{x} = n,\ \mathtt{y} = ([\![\tilde{\mathsf{c}}]\!]),\ \mathtt{w} = (\mathsf{c}) \mid \forall i,\ \mathsf{a}[i] = 1 \implies \mathsf{c}[i] \neq 0 \,\}.$$

A field element is nonzero exactly when it has a multiplicative inverse. The prover sends an inverse column inv whose value at each active index is the inverse of the data there and which is 0 at inactive indices. If inv is formed this way, then $\mathsf{d} \cdot \mathsf{inv} = \mathsf{a}$ holds across the whole hypercube: at an active index both sides equal 1, and at an inactive index both sides equal 0 (since inv is 0 there). A single Zerocheck on $\mathsf{d} \cdot \mathsf{inv} - \mathsf{a}$ certifies this, and forces every active entry to be invertible, hence nonzero.

PIOP 12: NoZero Check

Inputs: $[\![\tilde{\mathsf{c}}]\!] = ([\![\tilde{\mathsf{a}}]\!], [\![\tilde{\mathsf{d}}]\!])$.

1. $\mathbf{P}$ constructs and sends the inverse oracle $[\![\tilde{\mathsf{inv}}]\!]$, with $\mathsf{inv}[i] = \mathsf{d}[i]^{-1}$ at active indices and $\mathsf{inv}[i] = 0$ elsewhere.
2. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $\mathsf{d} \cdot \mathsf{inv} - \mathsf{a}$.

B.2 Booleanity Check

The *Booleanity Check* takes a column $\mathsf{c} = (\mathsf{d}, \mathsf{a})$ and proves that every active entry of c is 0 or 1. We package this as the relation

$$\mathscr{R}_{\mathsf{Bool}} = \{\, \mathbb{x} = \bot,\ \mathbb{y} = ([\![\tilde{\mathsf{c}}]\!]),\ \mathbb{w} = (\mathsf{c}) \mid \forall i,\ \mathsf{a}[i] = 1 \implies \mathsf{d}[i] \in \{0,1\} \,\}\,.$$

Since $\mathsf{d}[i] \in \{0,1\}$ exactly when $\mathsf{d}[i]\big(1-\mathsf{d}[i]\big) = 0$, the check is a single Zerocheck on the column $\mathsf{a}\cdot\mathsf{d}\cdot(1-\mathsf{d})$, which vanishes everywhere iff every active entry is boolean.

PIOP 13: Booleanity Check

Inputs: $[\![\tilde{\mathsf{c}}]\!] = ([\![\tilde{\mathsf{a}}]\!], [\![\tilde{\mathsf{d}}]\!])$.

1. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $\mathsf{a} \cdot \mathsf{d} \cdot (1 - \mathsf{d})$.

B.3 No-Duplicate Check

The *No-Duplicate Check* takes a column $\mathsf{c} = (\mathsf{d}, \mathsf{a})$ and proves that the active entries of c are pairwise distinct. We package this as the relation

$$\mathscr{R}_{\mathsf{NoDup}} = \{\, \mathbb{x} = \bot,\ \mathbb{y} = ([\![\tilde{\mathsf{c}}]\!]),\ \mathbb{w} = (\mathsf{c}) \mid \forall i \neq j,\ \mathsf{a}[i] = \mathsf{a}[j] = 1 \implies \mathsf{d}[i] \neq \mathsf{d}[j] \,\}\,.$$

PIOP 14: No-Duplicate Check

TODO

B.4 Limb Reconstruction Check

The *Limb Reconstruction Check* takes 1 n-bit column c and k columns $\mathsf{c}_0, \ldots, \mathsf{c}_{k-1}$ each with n/k bits and proves that each entry of c is the concatenation of the corresponding entries in $\mathsf{c}_0, \ldots, \mathsf{c}_{k-1}$. We package this as the relation

$$\mathscr{R}_{\mathsf{Limb}} = \left\{ \mathbb{x} = n,\ \mathbb{y} = ([\![\tilde{\mathsf{c}}]\!], [\![\tilde{\mathsf{c}_0}]\!], \ldots, [\![\tilde{\mathsf{c}_{k-1}}]\!]),\ \mathbb{w} = (\mathsf{c}, \mathsf{c}_0, \ldots, \mathsf{c}_{k-1}) \;\middle|\; \forall i,\ \mathsf{c}[i] = \sum_{j=0}^{k-1} 2^{(k-1-j)n/k} \mathsf{c}_j[i] \right\}.$$

Reconstruction is a single linear identity over the hypercube: subtracting the weighted limbs from c must vanish at every row. A single Zerocheck on $\mathsf{c} - \sum_{j=0}^{k-1} 2^{(k-1-j)n/k} \cdot \mathsf{c}_j$ enforces exactly this, pinning down each entry of c as the concatenation of the corresponding limbs. The check certifies only the weighted-sum relationship; bounding each limb c_j to its n/k-bit width, when required, is done by a separate Range Check.

PIOP 15: Limb Reconstruction Check

Inputs: $[\![\tilde{\mathsf{c}}]\!], [\![\tilde{\mathsf{c}_0}]\!], \ldots, [\![\tilde{\mathsf{c}_{k-1}}]\!]$.

1. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $\mathsf{c} - \sum_{j=0}^{k-1} 2^{(k-1-j)n/k} \cdot \mathsf{c}_j$.

B.5 Lookup Check

The *Lookup Check* takes two columns $\mathsf{c}_1 = (\mathsf{d}_1, \mathsf{a}_1)$ and $\mathsf{c}_2 = (\mathsf{d}_2, \mathsf{a}_2)$ and proves that every active value of c_1 also occurs as an active value of c_2, written $\mathsf{c}_1 \sqsubseteq \mathsf{c}_2$. We package this as the relation

$$\mathscr{R}_{\mathsf{Lookup}} = \{\, \mathbb{x} = \bot,\ \mathbb{y} = ([\![\tilde{\mathsf{c}_1}]\!], [\![\tilde{\mathsf{c}_2}]\!]),\ \mathbb{w} = (\mathsf{c}_1, \mathsf{c}_2) \mid \forall i,\ \mathsf{a}_1[i] = 1 \implies \exists j :\ \mathsf{a}_2[j] = 1 \wedge \mathsf{d}_1[i] = \mathsf{d}_2[j] \,\}\,.$$

We discharge the lookup with the LogUp technique [Hab22]: $\mathsf{c}_1 \sqsubseteq \mathsf{c}_2$ holds if and only if there is a *multiplicity* column m over c_2's hypercube such that, with μ and ν the hypercube dimensions of $\tilde{\mathsf{c}_1}$ and $\tilde{\mathsf{c}_2}$,

$$\sum_{x \in \mathscr{H}_\mu} \frac{\tilde{\mathsf{a}_1}(x)}{X - \tilde{\mathsf{d}_1}(x)} \;=\; \sum_{y \in \mathscr{H}_\nu} \frac{\tilde{\mathsf{a}_2}(y) \cdot \tilde{\mathsf{m}}(y)}{X - \tilde{\mathsf{d}_2}(y)} \tag{4}$$

as rational functions in X, where the activators zero out the inactive terms on each side. The prover sends m, and the parties verify the rational identity by Schwartz–Zippel: the verifier samples a random $r \xleftarrow{\$} \mathbb{F}$, the prover sends the purported common value s that both sides take at $X = r$, and the parties run RatSumcheck twice to confirm that each side evaluates to s at $X = r$. When c_2 is known to be duplicate-free, $\mathsf{m}(y)$ records the multiplicity of $\tilde{\mathsf{d}_2}(y)$ in c_1, so the verifier stores $[\![\tilde{\mathsf{m}}]\!]$ as a protocol output for use by higher-level checks.

PIOP 16: Lookup Check

Inputs: $[\![\tilde{\mathsf{c}_1}]\!], [\![\tilde{\mathsf{c}_2}]\!]$.

1. $\mathbf{P}$ constructs and sends the multiplicity oracle $[\![\tilde{\mathsf{m}}]\!]$, which $\mathbf{V}$ stores as protocol output.
2. $\mathbf{V}$ samples a random point $r \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathbf{P}$, who responds with the claimed common value s of both sides of Eq. (4) at $X = r$.
3. $\mathbf{P}$ and $\mathbf{V}$ invoke RatSumcheck twice to prove that both sides of Eq. (4) evaluate to s at $X = r$.

B.6 Range Check

The *Range Check* takes a column $\mathsf{c} = (\mathsf{d}, \mathsf{a})$ and a width n, and proves that every active entry of c is a non-negative integer of bitwidth n, i.e. lies in $[0, 2^n - 1]$. We package this as the relation

$$\mathscr{R}_{\mathsf{Range}} = \{\, \mathbb{x} = n,\ \mathbb{y} = ([\![\tilde{\mathsf{c}}]\!]),\ \mathbb{w} = (\mathsf{c}) \mid \forall i,\ \mathsf{a}[i] = 1 \implies 0 \le \mathsf{d}[i] < 2^n \,\}\,.$$

We assume $n < \log_2 |\mathbb{F}|$, so the range embeds unambiguously into $\mathbb{F}$. A direct approach proves $\mathsf{c} \sqsubseteq \mathsf{rng}_n := (\mathbf{1}, \mathcal{I}_n)$, since the index polynomial $\mathcal{I}_n$ ranges over exactly $\{0, \ldots, 2^n - 1\}$; but this lookup table has 2^n rows and is wasteful for large n. Instead we split each value into 16-bit *limbs* and range-check each limb against a small index column. Choosing k and β with $n = 16k + \beta$ and $\beta \le 16$, the prover sends limb columns $\mathsf{d}_1, \ldots, \mathsf{d}_{k+1}$ satisfying

$$\mathsf{a}(x) \cdot \mathsf{d}(x) = \sum_{i=1}^{k+1} 2^{16(i-1)} \cdot \mathsf{d}_i(x), \qquad x \in \mathscr{H}_\mu,$$

which a single Zerocheck enforces; the activator collapses inactive rows to zero, so the identity pins the limb decomposition on active rows. A Lookup Check of each limb into the appropriate small index column then forces $\mathsf{d}(x) \in [0, 2^n - 1]$ at every active x.

PIOP 17: Range Check

Inputs: $[\![\tilde{\mathsf{c}}]\!] = ([\![\tilde{\mathsf{a}}]\!], [\![\tilde{\mathsf{d}}]\!])$, width n.

1. $\mathbf{P}$ and $\mathbf{V}$ compute β, k such that $16k + \beta = n$ and $\beta \leq 16$.
2. $\mathbf{P}$ constructs and sends the limb oracles $[\![\tilde{\mathsf{d}_1}]\!], \ldots, [\![\tilde{\mathsf{d}_{k+1}}]\!]$, the 16-bit limb decomposition of d.
3. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck on the column $\mathsf{a} \cdot \mathsf{d} - \sum_{i=1}^{k+1} 2^{16(i-1)} \cdot \mathsf{d}_i$.
4. For each $i \in [k]$, $\mathbf{P}$ and $\mathbf{V}$ invoke Lookup Check on $(\mathbf{1}, \mathsf{d}_i)$ and $\mathsf{rng}_{16} := (\mathbf{1}, \mathcal{I}_{16})$, proving each limb lies in $[0, 2^{16} - 1]$.
5. $\mathbf{P}$ and $\mathbf{V}$ invoke Lookup Check on $(\mathbf{1}, \mathsf{d}_{k+1})$ and $\mathsf{rng}_\beta := (\mathbf{1}, \mathcal{I}_\beta)$, proving the residual limb lies in $[0, 2^\beta - 1]$.

B.7 Sign Check

The *Sign Check* takes a column $\mathsf{c} = (\mathsf{d}, \mathsf{a})$, a width α, and a sign mode $\sigma \in \{\mathsf{NonNeg}, \mathsf{Pos}, \mathsf{NonPos}, \mathsf{Neg}\}$, and proves that every active entry of c lies in the width-α slice of the corresponding sign. Each mode is determined by two parameters: a *direction* $s_\sigma \in \{+1, -1\}$, which is $+1$ for the positive-side modes $\mathsf{NonNeg}, \mathsf{Pos}$ and -1 for the negative-side modes $\mathsf{NonPos}, \mathsf{Neg}$; and a lower bound $\ell_\sigma \in \{0, 1\}$, which is 0 for the non-strict modes $\mathsf{NonNeg}, \mathsf{NonPos}$ and 1 for the strict modes $\mathsf{Pos}, \mathsf{Neg}$ that exclude zero. Concretely,

σ	NonNeg	Pos	NonPos	Neg
certified range	$[0, 2^\alpha - 1]$	$[1, 2^\alpha - 1]$	$[-(2^\alpha - 1), 0]$	$[-(2^\alpha - 1), -1]$
(s_σ, ℓ_σ)	$(+1, 0)$	$(+1, 1)$	$(-1, 0)$	$(-1, 1)$

where negative values are taken in $\mathbb{F}$ via the additive inverse. In every mode the certified condition is $s_\sigma \cdot \mathsf{d}[i] \in [\ell_\sigma, 2^\alpha - 1]$, so we package the check as the relation

$$\mathscr{R}_{\mathsf{Sign}} = \{\, \mathbb{x} = (n, \alpha, \sigma),\ \mathbb{y} = ([\![\tilde{\mathsf{c}}]\!]),\ \mathbb{w} = (\mathsf{c}) \mid \forall i,\ \mathsf{a}[i] = 1 \implies \ell_\sigma \leq s_\sigma \cdot \mathsf{d}[i] \leq 2^\alpha - 1 \,\}.$$

We assume $\alpha < \log_2 |\mathbb{F}|$, so the range embeds unambiguously into $\mathbb{F}$.

The whole check reduces to a non-negativity check on the *magnitude* column $\mathsf{m} := s_\sigma \cdot \mathsf{d}$, which the verifier obtains by negating the oracle $\tilde{\mathsf{d}}$ in the negative-side modes. The index polynomial $\mathcal{I}_\alpha$, evaluated over $\mathscr{H}_\alpha$, takes exactly the values $\{0, 1, \ldots, 2^\alpha - 1\}$. Treating it as the data of an α-variable column with the all-ones activator gives a range column $\mathsf{rng}_\alpha := (\mathbf{1}, \mathcal{I}_\alpha)$ whose value multiset is precisely $[0, 2^\alpha - 1]$, so a single Lookup Check of m into rng_α is already an exact range check. This naive lookup is wasteful when α is large, since rng_α has 2^α rows. We do better by splitting each entry into 16-bit *limbs* and range-checking each limb against a small index column. Choosing k and β with $\alpha = 16k + \beta$ and $\beta \leq 16$, the prover sends limb columns $\mathsf{m}_1, \ldots, \mathsf{m}_{k+1}$ satisfying

$$\mathsf{a}(x) \cdot s_\sigma \cdot \mathsf{d}(x) = \sum_{i=1}^{k+1} 2^{16(i-1)} \cdot \mathsf{m}_i(x), \qquad x \in \mathscr{H}_\mu,$$

which a single Zerocheck enforces; the activator on the left collapses inactive rows to zero, so the identity pins down the limb decomposition on active rows and forces the limbs to combine to zero on inactive rows. Range-checking each limb then pins $s_\sigma \cdot \mathsf{d}(x) \in [0, 2^\alpha - 1]$ at every active x. In the strict modes Pos and Neg, a final no-zero check rules out the value 0, narrowing the certified range to $[1, 2^\alpha - 1]$.

PIOP 18: Sign Check

Inputs: $[\![\tilde{\mathsf{c}}]\!] = ([\![\tilde{\mathsf{a}}]\!], [\![\tilde{\mathsf{d}}]\!])$, width α, sign mode $\sigma \in \{\mathsf{NonNeg}, \mathsf{Pos}, \mathsf{NonPos}, \mathsf{Neg}\}$.

1. $\mathbf{P}$ and $\mathbf{V}$ read off $s_\sigma \in \{+1, -1\}$ and $\ell_\sigma \in \{0, 1\}$ from σ, and compute β, k such that $16k + \beta = \alpha$ and $\beta \leq 16$.
2. $\mathbf{P}$ constructs and sends the limb oracles $[\![\tilde{\mathsf{m}_1}]\!], \ldots, [\![\tilde{\mathsf{m}_{k+1}}]\!]$ of the magnitude $\mathsf{m} = s_\sigma \cdot \mathsf{d}$.
3. $\mathbf{P}$ and $\mathbf{V}$ invoke Zerocheck to prove $\mathsf{a} \cdot s_\sigma \cdot \mathsf{d} - \sum_{i=1}^{k+1} 2^{16(i-1)} \cdot \mathsf{m}_i = 0$.
4. For each $i \in [k]$, $\mathbf{P}$ and $\mathbf{V}$ invoke Lookup Check on $(\mathbf{1}, \mathsf{m}_i)$ and $\mathsf{rng}_{16} := (\mathbf{1}, \mathcal{I}_{16})$, proving each limb lies in $[0, 2^{16} - 1]$.
5. $\mathbf{P}$ and $\mathbf{V}$ invoke Lookup Check on $(\mathbf{1}, \mathsf{m}_{k+1})$ and $\mathsf{rng}_\beta := (\mathbf{1}, \mathcal{I}_\beta)$, proving the residual limb lies in $[0, 2^\beta - 1]$.
6. If $\ell_\sigma = 1$ (the strict modes $\mathsf{Pos}, \mathsf{Neg}$), $\mathbf{P}$ and $\mathbf{V}$ additionally invoke NoZero Check on c to rule out zero entries, narrowing the certified range to $[1, 2^\alpha - 1]$.

B.8 Keyed Sumcheck

The *Keyed Sumcheck* takes two tables L and R, each with a key column key, a value column val, and an activator column a, and proves that for every possible key $\kappa \in \mathbb{F}$, the sum of active values in L with key κ equals the sum of active values in R with key κ. Define the keyed sums

$$S_{\mathsf{L}}(\kappa) := \sum_{\substack{i \in [|\mathsf{L}|] \\ \mathsf{L.key}[i] = \kappa}} \mathsf{L.a}[i] \cdot \mathsf{L.val}[i] \qquad \text{and} \qquad S_{\mathsf{R}}(\kappa) := \sum_{\substack{j \in [|\mathsf{R}|] \\ \mathsf{R.key}[j] = \kappa}} \mathsf{R.a}[j] \cdot \mathsf{R.val}[j].$$

We package this as the relation

$$\mathscr{R}_{\mathsf{ksum}} = \left\{ \mathbb{x} = \bot,\ \mathbb{y} = ([\![\tilde{\mathsf{L}}]\!], [\![\tilde{\mathsf{R}}]\!]),\ \mathbb{w} = (\mathsf{L}, \mathsf{R}) \;\middle|\; \forall\, \kappa \in \mathbb{F} : S_{\mathsf{L}}(\kappa) = S_{\mathsf{R}}(\kappa) \right\}.$$

KeyedSumcheck reduces the per-key equality $\forall \kappa : S_{\mathsf{L}}(\kappa) = S_{\mathsf{R}}(\kappa)$ to a single rational identity. The protocol is a generalization of the LogUp lookup protocol [Hab22], and we derive its security from a core lemma about function equality that is implicit in LogUp.

Lemma 1 (function equality [Hab22])**.** *For any functions $S_1, S_2 : \mathbb{F} \to \mathbb{F}$, $S_1(\kappa) = S_2(\kappa)$ for every $\kappa \in \mathbb{F}$ if and only if*

$$\sum_{\kappa \in \mathbb{F}} \frac{S_1(\kappa)}{X - \kappa} = \sum_{\kappa \in \mathbb{F}} \frac{S_2(\kappa)}{X - \kappa}$$

as rational functions in the formal variable X.

Applying Lemma 1 to S_{L} and S_{R} and collapsing the per-key double sums into single sums over rows (the per-key index sets partition the rows), the property holds if and only if, with μ and ν the hypercube dimensions of $\tilde{\mathsf{L}}$ and $\tilde{\mathsf{R}}$,

$$\sum_{x \in \mathscr{H}_\mu} \frac{\tilde{\mathsf{L}}.\mathsf{a}(x) \cdot \tilde{\mathsf{L}}.\mathsf{val}(x)}{X - \tilde{\mathsf{L}}.\mathsf{key}(x)} = \sum_{x \in \mathscr{H}_\nu} \frac{\tilde{\mathsf{R}}.\mathsf{a}(x) \cdot \tilde{\mathsf{R}}.\mathsf{val}(x)}{X - \tilde{\mathsf{R}}.\mathsf{key}(x)}. \tag{5}$$

We verify this rational identity by Schwartz–Zippel: the verifier samples a random $r \xleftarrow{\$} \mathbb{F}$, the prover sends the purported common value s that both sides take at $X = r$, and the parties run RatSumcheck twice to confirm that both sides actually evaluate to s at $X = r$.

PIOP 19: KeyedSumcheck

Inputs: $[\![\tilde{\mathsf{L}}]\!], [\![\tilde{\mathsf{R}}]\!]$.

1. $\mathbf{V}$ samples a random point $r \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathbf{P}$.
2. $\mathbf{P}$ computes and sends $\mathbf{V}$ the value s, the purported evaluation of both sides of Eq. (5) at $X = r$.
3. $\mathbf{P}$ and $\mathbf{V}$ invoke RatSumcheck twice to prove that both sides of Eq. (5) evaluate to s at $X = r$.

B.9 Multiset Equality Check

The *Multiset Equality Check* takes two columns $\mathsf{c_L}$ and $\mathsf{c_R}$ and proves that they hold the same multiset of active values. We package this as the relation

$$\mathscr{R}_{\mathsf{mset}} = \{\, \mathbb{x} = \bot,\ \mathbb{y} = ([\![\tilde{\mathsf{c_L}}]\!], [\![\tilde{\mathsf{c_R}}]\!]),\ \mathbb{w} = (\mathsf{c_L}, \mathsf{c_R}) \mid \{\!\{\, \mathsf{c_L}[i] \,\}\!\}_i = \{\!\{\, \mathsf{c_R}[j] \,\}\!\}_j \,\}\,.$$

Two multisets agree exactly when every value $\kappa \in \mathbb{F}$ occurs with the same multiplicity on both sides, i.e. $\forall\, \kappa \in \mathbb{F} : \sum_{i:\, \mathsf{c_L}[i]=\kappa} 1 = \sum_{j:\, \mathsf{c_R}[j]=\kappa} 1$. This is precisely an κ-keyed sum equality between two tables whose key is the column data and whose value is the constant $\mathbf{1}$. The parties locally form the auxiliary tables

$$\mathsf{L} := \big(\mathsf{a} = \mathsf{c_L}.\mathsf{a},\ \mathsf{key} = \mathsf{c_L}.\mathsf{data},\ \mathsf{val} = \mathbf{1}\big) \quad \text{and} \quad \mathsf{R} := \big(\mathsf{a} = \mathsf{c_R}.\mathsf{a},\ \mathsf{key} = \mathsf{c_R}.\mathsf{data},\ \mathsf{val} = \mathbf{1}\big),$$

and a single KeyedSumcheck on L and R closes out the proof, with completeness and soundness inherited from the keyed sumcheck.

PIOP 20: Multiset Equality Check

Inputs: $[\![\tilde{\mathsf{c_L}}]\!], [\![\tilde{\mathsf{c_R}}]\!]$.

1. $\mathbf{P}$ and $\mathbf{V}$ locally form the tables L and R as above.
2. $\mathbf{P}$ and $\mathbf{V}$ invoke KeyedSumcheck on L and R.

Handling tables. The foregoing column protocol extends to prove that two *tables* hold the same multiset of *rows*: the parties first fingerprint each table to a single column with a verifier-chosen random linear combination of its data columns, and then invoke the foregoing check on the two fingerprint columns.

B.10 Rotation Check

The *Rotation Check* takes two columns c and c' together with a shift $t \in \mathbb{Z}$, and proves that c' is the cyclic rotation of c by t positions, written $\mathsf{c}' = \rho_t(\mathsf{c})$, where ρ_t sends the entry at index p to index $p + t$ modulo the column length s. We package this as the relation

$$\mathscr{R}_{\mathsf{rot}} = \left\{ \mathbb{x} = t,\ \mathbb{y} = ([\![\tilde{\mathsf{c}}]\!], [\![\tilde{\mathsf{c}'}]\!]),\ \mathbb{w} = (\mathsf{c}, \mathsf{c}') \;\middle|\; \begin{array}{l} |\mathsf{c}| = |\mathsf{c}'| =: s \\ \forall\, q \in [s] : \mathsf{c}'[q] = \mathsf{c}[(q - t) \bmod s] \end{array} \right\}.$$

We first adopt the contiguous convention so that the active rows of both columns occupy hypercube indices $0, \ldots, s-1$, which the verifier enforces by asserting $\mathsf{c.a} = \mathsf{c}'.\mathsf{a} = \mathcal{C}_{\mu,s}$. We then reduce the rotation condition to a single multiset equality on *position-tagged* columns: tag each entry of c with its own position, and tag each entry of c' with the position it should occupy in c. Concretely, the parties locally construct the tagged tables

$$\mathsf{T} := \big(\mathsf{c.a},\ \mathsf{c.data},\ \mathcal{I}_\mu\big) \quad \text{and} \quad \mathsf{T}' := \big(\mathsf{c}'.\mathsf{a},\ \mathsf{c}'.\mathsf{data},\ \mathcal{I}_\mu^{\mathsf{rot}}\big),$$

where $\mathcal{I}_\mu$ maps each $x \in \mathscr{H}_\mu$ to its hypercube position and $\mathcal{I}_\mu^{\mathsf{rot}}$ is the *shifted* index polynomial sending each active position q to $(q - t) \bmod s$. Writing $t_0 := t \bmod s$, this shift is realized exactly by

$$\mathcal{I}_\mu^{\mathsf{rot}}(x) := \mathcal{I}_\mu(x) - t_0 + s \cdot \mathcal{C}_{\mu,t_0}(x),$$

since $\mathcal{C}_{\mu,t_0}(x) = 1$ precisely when $\mathcal{I}_\mu(x) < t_0$, supplying the $+s$ wraparound exactly on the positions that underflow. Because $q \mapsto (q - t) \bmod s$ is a bijection on $\{0, \ldots, s-1\}$, the tagged tables agree as multisets of rows if and only if $\mathsf{c}'[q] = \mathsf{c}[(q - t) \bmod s]$ for every q, which is exactly the rotation condition. A single Multiset Equality Check on T and T' then proves the claim, with completeness and soundness inherited from the multiset equality check.

PIOP 21: Rotation Check

Inputs: $[\![\tilde{\mathsf{c}}]\!], [\![\tilde{\mathsf{c}'}]\!], t$.

1. $\mathbf{P}$ sends $\mathbf{V}$ the active length s, and $\mathbf{V}$ asserts $\mathsf{c.a} = \mathsf{c}'.\mathsf{a} = \mathcal{C}_{\mu,s}$.
2. $\mathbf{P}$ and $\mathbf{V}$ locally construct the tagged tables T and T' as above, using the shifted index polynomial $\mathcal{I}_\mu^{\mathsf{rot}}(x) = \mathcal{I}_\mu(x) - t_0 + s \cdot \mathcal{C}_{\mu,t_0}(x)$ with $t_0 := t \bmod s$.
3. $\mathbf{P}$ and $\mathbf{V}$ invoke Multiset Equality Check to prove that T and T' are equal as multisets of rows.

References

[BCGGRS19] E. Ben-Sasson, A. Chiesa, L. Goldberg, T. Gur, M. Riabzev, and N. Spooner. "Linear-Size Constant-Query IOPs for Delegating Computation". In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC '19. 2019.

[BFS20] B. Bünz, B. Fisch, and A. Szepieniec. "Transparent SNARKs from DARK Compilers". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020, pp. 677–706.

[BMMS25] A. Baweja, P. Mishra, T. Mopuri, and M. Shtepel. "FICS and FACS: Fast IOPPs and Accumulation via Code-Switching". Cryptology ePrint Archive, Paper 2025/737. 2025.

[BMNW25] B. Bünz, P. Mishra, W. Nguyen, and W. Wang. "Arc: Accumulation for Reed–Solomon Codes". In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO '25. 2025.

[CHMMVW20] A. Chiesa, Y. Hu, M. Maller, P. Mishra, N. Vesely, and N. Ward. "Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020.

[GWC19] A. Gabizon, Z. J. Williamson, and O. Ciobotaru. "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge". Cryptology ePrint Archive, Report 2019/953. 2019.

[Hab22] U. Haböck. "Multivariate lookups based on logarithmic derivatives". Cryptology ePrint Archive, Paper 2022/1530. `https://eprint.iacr.org/2022/1530`. 2022.

[KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. "Constant-Size Commitments to Polynomials and Their Applications". In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '10. 2010, pp. 177–194.

[NSSMM26] B. Namboothiry, A. Shirzad, S. Solit, R. Marcus, and P. Mishra. "TruthTable: A Verifiable Query Engine". Cryptology ePrint Archive, Paper 2026/1260. 2026.